디지털 시스템 설계 — 수업정리노트

Ch0: Introduction & Trend in Computer Systems

과목 개요

이 과목 “Digital Systems Design”은 서울대 심재웅 교수님이 삼성DS System Expert 과정을 위해 진행하는 수업이다. 핵심 목표는 CPU/GPU/TPU와 같은 작은 하드웨어 조각을 직접 설계할 수 있는 능력을 기르는 것이다.

왜 이 과목이 필요한가? 소프트웨어 엔지니어가 하드웨어의 동작 원리를 이해하면, 시스템 레벨에서의 최적화 판단이 가능해진다. 특히 삼성DS처럼 반도체를 설계하는 조직에서는 SW/HW 인터페이스를 이해하는 것이 핵심 역량이다.

과목에서 다루는 주제는 시스템 스택의 Logic 레벨에 해당한다:

```
Algorithms
  └ Prog. Model/Lang.
       └ System Software
            └ Compiler
                 └ SW/HW Interface
                      └ Micro-architecture
                           └ Logic          ◀◀◀ 여기!
                                └ Devices
                                     └ Physics
```

구체적 토픽: 조합/순차 논리 복습 → Verilog HDL → 수 체계 & 부동소수점 → 하드웨어 빌딩 블록(Adder, Multiplier, SRAM, DRAM) → DNN 소개 → FPGA로 CNN 가속기 구현

컴퓨터 시스템의 트렌드

프로세서 진화의 역사

1971년 Intel 4004 (2,300 트랜지스터, 0.74MHz)에서 시작하여 오늘날 540억 개 이상의 트랜지스터를 가진 Manycore 프로세서까지, 2천만 배 이상 증가했다. 그 과정에서 성능을 끌어올린 핵심 기법들은 시대별로 달랐다:

```
1971: 4004 (0.74MHz)
  │  ← Clock Speed 향상
1978: 8086 (10MHz)
  │  ← Bus Width 확장
1993: Pentium (200MHz, Superscalar)
  │  ← Pipeline Width 확장
2006: Core2Duo (3.3GHz, Dual Core)
  │  ← Core 수 증가
TODAY: Manycore
```

Moore’s Law (무어의 법칙)

Intel 공동창업자 Gordon Moore가 제안한 경험 법칙으로, 칩 위의 트랜지스터 수가 약 12~24개월마다 2배로 증가한다는 것이다. 이 법칙은 50년간 놀랍도록 잘 맞아왔지만, 현재는 둔화되고 있다.

핵심 직관: 트랜지스터가 작아지면 → 같은 면적에 더 많이 넣을 수 있고 → 더 복잡한 회로를 만들 수 있다.

Dennard Scaling (데나드 스케일링)

Bob Dennard (1974)가 관찰한 법칙: 트랜지스터가 작아져도 전력 밀도(power density)는 일정하게 유지된다.

핵심 수식:

```
P ∝ C · V² · F

여기서:
  P = 전력 (Power)
  C = 커패시턴스 (Capacitance) — 트랜지스터 크기에 비례
  V = 전압 (Voltage)
  F = 주파수 (Frequency)
```

트랜지스터가 작아지면 C가 줄어들고, V도 낮출 수 있어서, F를 올려도 전력이 비슷했다. 하지만 2000년대 중반 이후 전압을 더 이상 낮출 수 없게 되면서 Dennard Scaling은 끝났다. 이것이 주파수 경쟁이 멈추고 멀티코어 시대로 전환된 이유다.

오늘날의 도전과 해법

성능 향상의 원천을 분석하면 (AMD Lisa Su 발표 기준): - 공정 기술: 40% - 마이크로아키텍처: 17% - 전력 관리: 15% - 다이 크기 증가: 12% - 추가 TDP: 8% - 컴파일러: 8%

현재의 도전: Moore's Law 둔화 + Dennard Scaling 종료 + 다이 크기 레티클 한계(~800mm²) + 수율 좋은 mm²당 비용 증가

해법은 특수 목적 가속기(Specialized Accelerators)이다:

- Google TPU v1: 28nm, 331mm², 700MHz, 92 TOPs (INT8), 40W
- Google TPU v2/v3: HBM 탑재, bfloat16 학습 지원, 최대 105 TFLOPs
- Tesla FSD Chip: Samsung 14nm, 260mm², 36W, NPU 2개 (각 36.86 INT8 TOPs)
- AWS Inferentia: FP16/BFP16/INT8 지원, 최대 128 TOPS
- Intel Loihi: 뉴로모픽 칩, SNN(Spiking Neural Network) 전용, 14nm 60mm²
- Google 양자컴퓨팅 등

핵심 메시지: "A New Golden Age for Computer Architecture" — 하드웨어 플랫폼과 애플리케이션이 계속 진화하고 있으며, 시스템 스택 전체를 재설계할 필요가 있다. 하지만 그 전에 디지털 시스템의 기초(fundamentals)를 이해해야 한다.

Ch1: Review — Logic Design

이 챕터는 Verilog를 배우기 전에 디지털 논리 설계의 기초를 빠르게 복습하는 것이 목적이다. 조합 논리(Combinational Logic)와 순차 논리(Sequential Logic)를 모두 다룬다.

1. Logic Gates (논리 게이트)

논리 게이트는 하나 이상의 이진 입력을 받아 하나의 이진 출력을 생성하는 디지털 회로의 가장 기본 요소이다. 각 게이트는 세 가지로 설명된다: 심벌(회로 기호), 불리언 수식, 진리표.

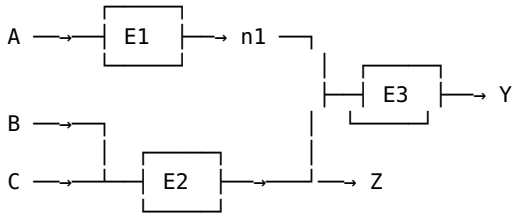
Gate	Boolean Eq.	Truth Table (A,B→Y)
BUF	$Y = A$	0→0, 1→1
NOT	$Y = \overline{A}$	0→1, 1→0
AND	$Y = A \cdot B$	00→0,01→0,10→0,11→1
OR	$Y = A+B$	00→0,01→1,10→1,11→1
XOR	$Y = A \oplus B$	00→0,01→1,10→1,11→0
NAND	$Y = \overline{(A \cdot B)}$	00→1,01→1,10→1,11→0
NOR	$Y = \overline{(A+B)}$	00→1,01→0,10→0,11→0

NOT 게이트의 출력 쪽에 있는 작은 원(bubble)은 반전(inversion)을 의미한다. NAND = AND + bubble, NOR = OR + bubble.

주의사항: XOR은 "둘 중 하나만 1일 때 1"이고, XNOR(= XOR + NOT)은 "둘 다 같을 때 1"이다. XOR은 패리티 검사, 가산기 등에서 핵심적으로 사용된다.

2. Digital Circuits (디지털 회로)

디지털 회로는 이산 값(discrete-valued) 변수를 처리하는 네트워크이다. 회로는 요소(elements)와 노드(nodes)로 구성된다.




```
입력 노드: A, B, C
출력 노드: Y, Z
내부 노드: n1
요소: E1, E2, E3
```

각 요소는 functional spec(입력→출력 관계)과 timing spec(지연 시간)으로 정의된다.

조합 회로 (Combinational): 출력이 현재 입력의 조합에만 의존. 메모리 없음.

순차 회로 (Sequential): 출력이 입력의 시퀀스(현재 + 이전 값)에 의존. 메모리 있음.

이 구분이 중요한 이유: 조합 회로는 같은 입력이면 항상 같은 출력을 내지만, 순차 회로는 과거 이력에 따라 출력이 달라진다. 현실의 거의 모든 유용한 디지털 시스템은 두 가지를 조합해서 만든다.

3. Boolean Equations & Terminology (불리언 수식과 용어)

불리언 수식은 TRUE(1) 또는 FALSE(0)인 변수들의 식으로, 디지털 논리를 수학적으로 기술하는 데 완벽하다. 진리표가 주어지면 불리언 수식을 쓸 수 있다.

Literal(리터럴): 변수 또는 그 보수. 예: A, $\bar{A}$, B, $\bar{B}$

Product(곱) = AND of literals. 예: A·B, $\bar{A}$·B·C Sum(합) = OR of literals. 예: A+B, $\bar{A}$+B+C

Implicant(함의항): 하나 이상의 리터럴의 AND. 예: A·B, $\bar{A}$·B·C, B 모두 3변수 함수의 implicant

Minterm(최소항): 함수의 모든 입력 변수를 포함하는 곱. 3변수 함수에서 $\bar{A}$·B·C는 minterm이지만, A·B는 C가 빠졌으므로 minterm이 아니다.

Maxterm(최대항): 함수의 모든 입력 변수를 포함하는 합. 3변수 함수에서 A+$\bar{B}$+C는 maxterm이지만, A+C는 B가 빠졌으므로 maxterm이 아니다.

A	B	C	Minterm	Name	Maxterm	Name
0	0	0	$\bar{A}\cdot\bar{B}\cdot\bar{C}$	m0	A+B+C	M0
0	0	1	$\bar{A}\cdot\bar{B}\cdot C$	m1	A+B+$\bar{C}$	M1
0	1	0	$\bar{A}\cdot B\cdot\bar{C}$	m2	A+$\bar{B}$+C	M2
0	1	1	$\bar{A}\cdot B\cdot C$	m3	A+$\bar{B}$+$\bar{C}$	M3
1	0	0	$A\cdot\bar{B}\cdot\bar{C}$	m4	$\bar{A}$+B+C	M4
1	0	1	$A\cdot\bar{B}\cdot C$	m5	$\bar{A}$+B+$\bar{C}$	M5
1	1	0	$A\cdot B\cdot\bar{C}$	m6	$\bar{A}$+$\bar{B}$+C	M6
1	1	1	A·B·C	m7	$\bar{A}$+$\bar{B}$+$\bar{C}$	M7

규칙: minterm에서 변수가 0이면 보수($\bar{A}$), 1이면 true form(A). Maxterm은 반대: 0이면 true form(A), 1이면 보수($\bar{A}$).

왜 “min”term과 “max”term인가? SOP(Sum of Products) = minterm들의 OR. OR의 최소 만족성: minterm 하나만 TRUE여도 출력이 TRUE → “min”. POS(Product of Sums) = maxterm들의 AND. AND의 최대 만족성: maxterm 전부가 TRUE여야 출력이 TRUE → “max”.

SOP Canonical Form: 진리표에서 출력 Y가 1인 행의 minterm들을 OR로 연결. 예: Y = $\bar{A}$·B + A·B, Σ(1, 3).

POS Canonical Form: 진리표에서 출력 Y가 0인 행의 maxterm들을 AND로 연결. 예: Y = (A+B)·($\bar{A}$+B), Π(0, 2).

주의: SOP와 POS는 같은 함수를 다른 형태로 표현한 것이다. 어느 쪽이 더 간단한지는 함수에 따라 다르다.

4. Boolean Algebra (불리언 대수)

진리표에서 직접 쓴 불리언 수식은 가장 간단한 형태가 아닐 수 있다. 불리언 대수를 이용하면 수식을 간소화할 수 있고, 간단한 수식 = 더 적은 게이트 = 더 작고 빠르고 에너지 효율적인 회로가 된다.

Duality(쌍대성): 불리언 대수의 모든 공리와 정리는 쌍대 원리를 따른다: 0↔1과 ·(AND)↔+(OR)를 서로 바꾸면 여전히 성립하는 식이 된다.

Axioms(공리):

	Axiom		Dual	Name
A1	B=0 if B≠1	A1'	B=1 if B≠0	Binary
A2	$\bar{0}$= 1	A2'	$\bar{1}$= 0	NOT
A3	0·0 = 0	A3'	1+1 = 1	AND/OR
A4	1·1 = 1	A4'	0+0 = 0	AND/OR
A5	0·1 = 1·0 = 0	A5'	1+0 = 0+1 = 1	AND/OR

Theorems of One Variable(단일 변수 정리):

	Theorem		Dual	Name
T1	$B \cdot 1 = B$	T1'	$B+0 = B$	Identity
T2	$B \cdot 0 = 0$	T2'	$B+1 = 1$	Null Element
T3	$B \cdot B = B$	T3'	$B+B = B$	Idempotency
T4	$\overline{\overline{B}} = B$			Involution
T5	$B \cdot \overline{B} = 0$	T5'	$B+\overline{B} = 1$	Complements

직관 설명: - Identity(T1): 1과 AND하면 자기 자신, 0과 OR하면 자기 자신 - Null(T2): 0과 AND하면 무조건 0, 1과 OR하면 무조건 1 - Idempotency(T3): 자기 자신끼리 AND/OR하면 자기 자신 - Involution(T4): 두 번 반전하면 원래대로 - Complements(T5): 자기 자신과 보수를 AND하면 0, OR하면 1

Theorems of Several Variables(다변수 정리):

	Theorem		Dual	Name
T6	$B \cdot C = C \cdot B$	T6'	$B+C = C+B$	Commutativity
T7	$(B \cdot C) \cdot D = B \cdot (C \cdot D)$	T7'	$(B+C)+D = B+(C+D)$	Associativity
T8	$(B \cdot C)+(B \cdot D) = B \cdot (C+D)$	T8'	$(B+C) \cdot (B+D) = B+(C \cdot D)$	Distributivity
T9	$B \cdot (B+C) = B$	T9'	$B+(B \cdot C) = B$	Covering
T10	$(B \cdot C)+(B \cdot \overline{C}) = B$	T10'	$(B+C) \cdot (B+\overline{C}) = B$	Combining
T11	$(B \cdot C)+(\overline{B} \cdot D)+(C \cdot D)$	T11'	$(B+C) \cdot (\overline{B}+D) \cdot (C+D)$	Consensus
	$= B \cdot C + \overline{B} \cdot D$		$= (B+C) \cdot (\overline{B}+D)$	
T12	$(B_0 \cdot B_1 \cdot B_2 \ldots)^{\overline{}}$	T12'	$(B_0+B_1+B_2 \ldots)^{\overline{}}$	De Morgan's
	$= \overline{B}_0+\overline{B}_1+\overline{B}_2 \ldots$		$= \overline{B}_0 \cdot \overline{B}_1 \cdot \overline{B}_2 \ldots$	

특히 중요한 정리들:

- T9 (Covering): $B \cdot (B+C) = B$. 직관: B가 이미 1이면 (B+C)도 반드시 1이므로 C는 무관.
- T10 (Combining): $B \cdot C + B \cdot \overline{C} = B$. 이것이 간소화의 핵심이다. C와 $\overline{C}$가 둘 다 있으면 C를 제거할 수 있다.
- T11 (Consensus): 세 번째 항($C \cdot D$)이 처음 두 항으로부터 파생되는 중복 항이므로 제거 가능.
- T12 (De Morgan's): AND의 반전 = 반전들의 OR, OR의 반전 = 반전들의 AND.

De Morgan's Theorem의 중요성:

NAND ≡ OR with inverted inputs: $(A \cdot B)^{\overline{}} = \overline{A} + \overline{B}$
NOR ≡ AND with inverted inputs: $(A+B)^{\overline{}} = \overline{A} \cdot \overline{B}$

이로부터 Logical Completeness(논리적 완전성): AND, OR, NOT만으로 모든 회로를 만들 수 있다. 더 나아가 NAND 게이트만으로(또는 NOR 게이트만으로) 모든 회로를 구현할 수 있다! 실제 반도체 제조에서 NAND/NOR 기반 설계가 많이 쓰이는 이유이다.

간소화 예제:

```
Ā·B̄·C + Ā·B·C + A·B̄·C 를 간소화:

단계 1: Ā·B̄·C + Ā·B·C                    (앞 두 항에서 B̄와 B로 T10 적용)
      = Ā·C·(B̄+B)  = Ā·C·1 = Ā·C

단계 2: Ā·C + A·B̄·C                       (C를 공통으로)
      = C·(Ā + A·B̄)
      = C·(Ā+A)·(Ā+B̄)                    (T8' 분배법칙)
      = C·1·(Ā+B̄) = C·(Ā+B̄)

결과: Y = Ā·C + B̄·C  또는 동치: Y = C·(Ā+B̄)
```

Prime Implicant(주요 함의항): Implicant가 다른 implicant와 더 이상 합쳐질 수 없을 때 prime implicant라고 한다. 최소화된 수식의 모든 항은 반드시 prime implicant여야 한다.

5. Karnaugh Maps (K-Map, 카르노 맵)

1953년 Bell Labs의 Maurice Karnaugh가 발명한 그래픽 기반의 불리언 수식 간소화 방법이다.

핵심 아이디어: 합칠 수 있는 항들(즉, 하나의 변수만 다른 항들)을 그리드에서 인접하게 배치한다. 이를 위해 Gray Code 순서를 사용한다 (00, 01, 11, 10 — 연속된 값이 1비트만 차이남).

2변수 K-Map:

```
        A
      0   1
    ┌───┬───┐
B 0 │ m0 │ m2 │
    ├───┼───┤
  1 │ m1 │ m3 │
    └───┴───┘
```

3변수 K-Map:

```
          AB
       00  01  11  10
    ┌────┬────┬────┬────┐
C  0│ m0 │ m2 │ m6 │ m4 │
    ├────┼────┼────┼────┤
   1│ m1 │ m3 │ m7 │ m5 │
    └────┴────┴────┴────┘
```

K-Map 간소화 규칙:

1. 모든 1을 포함하도록 가능한 한 적은 수의 사각형으로 묶는다
2. 각 사각형 안에는 1만 있어야 한다
3. 각 사각형은 2의 거듭제곱 크기의 직사각형이어야 한다 (1, 2, 4, 8, ...)
4. 각 사각형은 가능한 한 크게
5. K-Map의 가장자리는 연결된다 (좌우 wrap-around, 상하 wrap-around)
6. 하나의 1이 여러 사각형에 포함될 수 있다 (묶는 수를 줄이기 위해)

6. Combinational Building Blocks (조합 빌딩 블록)

Multiplexer (Mux, 멀티플렉서)

선택 신호(select signal)에 따라 여러 입력 중 하나를 출력으로 선택하는 회로.

2:1 Mux:

```
  D₀ ──┤ 0 │
       │   ├── Y
  D₁ ──┤ 1 │
       └───┘
         │
         S
```

수식: $Y = D_0 \cdot \overline{S} + D_1 \cdot S$

$S=0 \rightarrow Y=D_0$
$S=1 \rightarrow Y=D_1$

핵심 성질: 2^(N-1) 입력 Mux로 N 입력 논리 함수를 구현할 수 있다. Mux는 일종의 lookup table(LUT)로 동작한다. 이것이 FPGA의 기본 원리이기도 하다.

Decoder (디코더)

N개 입력, 2^N개 출력. 입력 조합에 따라 출력 하나만 활성화(assert). 디코더를 OR 게이트와 결합하면 모든 논리 함수를 구현할 수 있다. 각 출력이 하나의 minterm에 대응하기 때문이다.

X's and Z's

실제 회로에서는 0과 1 외에 두 가지 특수 값이 있다:

X (Illegal/Unknown value): 회로 노드의 값을 알 수 없는 상태. 시뮬레이터에서 초기화되지 않은 값을 나타낼 때도 사용.

Z (Floating/High-impedance value): 노드가 HIGH도 LOW도 아닌 상태. tri-state 버퍼의 출력이 비활성화될 때 나타남.

주의: X와 Z는 Verilog에서 매우 중요하게 사용된다. X는 "don't care"로 K-Map 최적화에서도 활용될 수 있다.

7. Sequential Circuits (순차 회로)

순차 논리는 메모리를 가진다. 이전의 입력을 명시적으로 기억하거나, 이전 입력을 시스템의 상태(state)로 축약한다.

Bistable Element (쌍안정 소자)

메모리의 가장 기본 빌딩 블록. 두 개의 안정 상태를 가진 소자이다.

인버터 두 개를 루프로 연결하면 간단한 bistable element를 만들 수 있다:

```
  ┌─────┐       ┌─────┐
Q─┤ I1  ├──→────┤ I2  ├──→──Q
  └─────┘       └─────┘
        ↑                  │
        └──────────────────┘
```

- Q=0이면: I2 입력=0 → I2 출력=1 → I1 입력=1 → I1 출력=0 → Q=0 (안정!)
- Q=1이면: I2 입력=1 → I2 출력=0 → I1 입력=0 → I1 출력=1 → Q=1 (안정!)

N개의 안정 상태를 가진 소자는 $\log_2 N$ 비트의 정보를 저장한다.

문제점: 상태를 바꿀 수 있는 입력이 없다! → Latch와 Flip-Flop이 필요.

8. Latches and Flip-Flops

Latch는 신호 레벨에 반응(CLK=1일 때 transparent)하고, Flip-Flop은 클럭 에지(전이)에 반응(에지 순간에만 값 갱신)한다.

SR (Set-Reset) Latch

두 개의 교차 결합된 NOR (또는 NAND) 게이트로 구성.

```
R ──→──┤ N1 ├──o──→ Q
       │    │     \
       └────┘      \
                    \
                     \
                     |
                     |
       ┌────┐        |
S ──→──┤ N2 ├──o──→ Q̄
       └────┘
```

진리표:

Case	S	R	Q	$\bar{Q}$	
IV	0	0	Qprev	$\bar{Q}$prev	(유지 — 메모리!)
I	0	1	0	1	(Reset)
II	1	0	1	0	(Set)
III	1	1	0	0	(금지! Q와 $\bar{Q}$ 모순)

함정: S=R=1은 Q=$\bar{Q}$=0이 되어 "Q와 $\bar{Q}$이 서로 보수"라는 전제가 깨진다. 이 상태에서 S와 R을 동시에 0으로 놓으면 출력이 불확정(race condition)이 된다.

D Latch

SR Latch의 두 가지 문제를 해결: 1. S=R=1 상태가 발생하지 않도록 → 데이터 입력 D 하나로 S와 R을 자동 생성 (S=D, R=$\bar{D}$) 2. 무엇을(D) 바꿀지와 언제(CLK) 바꿀지를 분리

CLK=1: transparent (Q follows D). CLK=0: opaque (Q holds previous value).

주의사항: D Latch는 CLK=1인 동안 계속 D를 통과시킨다(transparent).

D Flip-Flop

두 개의 D Latch를 back-to-back으로 연결 (master + slave) + 상보적 클럭.

```
                  CLK
                   |
                ┌──o──┐
                |     |
       CLK      |     |   CLK
D ──→──┤ D  Q ├──→N1──→┤ D  Q ├──→ Q
```

```
        │ L1 Q̄│      │ L2 Q̄├─→ Q̄
        └──────┘      └──────┘

         master        slave
```

동작: - CLK=0: master transparent (D → N1), slave opaque (N1 차단) - CLK=1: master opaque (D 차단), slave transparent (N1 → Q) - 결과: rising edge (0→1)에서만 D의 값이 Q로 복사되고, 나머지 시간에는 값 유지

이것이 디지털 회로에서 가장 핵심적인 저장 소자인 이유: 클럭 에지에서만 값이 바뀌므로 timing을 정확하게 제어할 수 있다.

Register(레지스터): 공통 CLK을 공유하는 N개 flip-flop의 뱅크. 대부분 순차 회로의 핵심 빌딩 블록이다.

9. Synchronous vs Asynchronous

동기식(Synchronous): 상태 전이가 클럭 에지에서만 발생. 거의 모든 실용적 디지털 시스템. 비동기식(Asynchronous): 클럭 없이 입력 변화에 즉시 반응. 설계가 매우 어렵다. 예: Ring Oscillator.

10. Finite State Machines (FSMs)

FSM은 k개의 레지스터를 가진 동기식 순차 회로로, 최대 2^k개의 고유 상태 중 하나에 있을 수 있다.

FSM = Next State Logic + State Register + Output Logic.

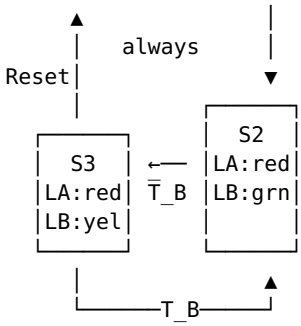
```
Moore Machine:
                          CLK
                           │
  inputs ─→┤ Next State ├─→┤ Register ├─→┤ Output Logic ├─→ outputs
           │   Logic    │   │          │   │              │
           └────────────┘   └──────────┘   └──────────────┘

  출력 = f(현재 상태만)

Mealy Machine:
                          CLK
                           │
  inputs ─→┤ Next State ├─→┤ Register ├─→┤ Output Logic ├─→ outputs
           │   Logic    │   │          │   │      ↑       │
           └────────────┘   └──────────┘   │   inputs     │
                                           └──────────────┘

  출력 = f(현재 상태 + 현재 입력)
```

Moore: 출력이 안정적, 한 클럭 늦게 반응. Mealy: 더 빠르게 반응, 입력 glitch에 민감.

FSM 설계 절차: State Transition Diagram → State Transition Table → State Encoding → Boolean Equations → Schematic.

Traffic Light Controller 예제:

```
          T_A
     ┌──────────┐
     │          │
     ▼   T̄_A    │
  ┌──────┐ ──→ ┌──────┐
  │  S0  │     │  S1  │
  │LA:grn│     │LA:yel│
  │LB:red│     │LB:red│
  └──────┘     └──────┘
```



State Encoding: S0=00, S1=01, S2=10, S3=11. Output: green=00, yellow=01, red=10.

Next State: $S'_1 = S_1 \oplus S_0$, $S'_0 = \bar{S}_1 \cdot \bar{S}_0 \cdot \bar{T}_A + S_1 \cdot \bar{S}_0 \cdot \bar{T}_B$. Output: $L_A1 = S_1$, $L_A0 = \bar{S}_1 \cdot S_0$, $L_B1 = \bar{S}_1$, $L_B0 = S_1 \cdot S_0$.

이것은 Moore machine이다. 출력이 현재 상태(S_1, S_0)에만 의존하고, 입력(T_A, T_B)에 직접 의존하지 않기 때문이다.

Ch2: Introduction to Verilog

이 챕터는 Verilog HDL의 기초를 다룬다. 여기서 가장 중요한 세 가지를 먼저 짚으면: (1) wire vs reg의 차이, (2) Continuous Assignment vs Procedural Assignment의 차이, (3) 세 가지 모델링 레벨 (Structural/Dataflow/Behavioral)의 용도와 차이이다.

1. Hardware Description Language (HDL)이란?

하드웨어를 설계할 때 회로도(schematic)를 직접 그리는 방법이 있지만, 수백만~수십억 개의 트랜지스터를 가진 현대 칩에서는 불가능하다. 텍스트 기반으로 하드웨어를 기술할 수 있는 언어가 필요한데, 그것이 HDL이다.

Verilog: 1984년 Gateway Design Automation에서 개발, 1990년 Cadence에 인수. 1995년 IEEE 1364 표준이 됨. 버전: Verilog-95 → Verilog-2001 → Verilog-2005 → SystemVerilog (업계 표준).

VHDL (Very High Speed Integrated Circuit HDL): 1983년 미국 국방부에서 개발. 1987년 IEEE 1076 표준.

이 과목에서는 Verilog를 사용한다.

SW Programming vs HW Description — 핵심 차이

이것이 Verilog를 배울 때 가장 중요한 직관이다:

소프트웨어: 주어진 하드웨어 위에서 명령문을 순서대로 실행. HDL: 주어진 회로를 텍스트로 기술 — 모든 것이 본질적으로 동시에(concurrently) 동작.

```
// 이것은 "a를 x에 복사하라"가 아니라
// "a에서 x로 가는 와이어 연결이 있다"는 의미!
assign x = a;
assign y = b;
// 두 assign문은 동시에 동작한다 (순서 무관)
```

assign은 RHS에서 LHS로 데이터가 흐르는 연결(connection)을 기술하는 것이지, RHS를 LHS로 복사하는 동작이 아니다.

Synthesizable Verilog

합성(Synthesis): HDL 코드 → Logic Gates & Wires (netlist) 변환.

Verilog 코드를 하드웨어로 매핑하려면 합성 가능(synthesizable)해야 한다. 모든 Verilog 구문이 합성 가능한 것은 아니다:

```
합성 가능:  module, instantiation, always, assign, wire, reg ...
합성 불가:  initial, $display, $monitor, #delay, $finish ...
            (이것들은 시뮬레이션/테스트벤치 전용)
```

주의: initial 블록은 합성 불가능하다! 테스트벤치에서만 사용. always 블록은 합성 가능하다(reg 타입 할당 시).

2. Module — Verilog의 기본 단위

Module은 Verilog에서 가장 핵심적인 빌딩 블록이다. 입력과 출력을 가진 모든 하드웨어 블록은 모듈이다 (AND 게이트, Mux, 디코더, CPU 전체 등).

모듈은 두 부분으로 구성된다: - Core circuit (internal/body): 필요한 기능을 수행하는 내부 회로 - Ports (interface): 내부 회로와 외부 세계를 연결하는 인터페이스

```
  ┌──────────────────────────┐
  │          Module          │
  │  ┌────────────────────┐  │
a─┼─→│   Core Circuit     │  │
  │  │   (Internal/Body)  │─→┼─→ y
b─┼─→│                    │  │
  │  └────────────────────┘  │
  │       Ports (Interface)  │
  └──────────────────────────┘
```

```
module test (input  a,
             input  b,
             output y);
  // describe our circuit here!
endmodule
```

주의: module 선언 끝의 세미콜론(;)을 잊지 말 것! endmodule 뒤에는 세미콜론이 없다.

Port Declaration: ANSI-C 스타일(Verilog-2001)을 사용하라. 읽기 쉽고 타이핑 실수가 적다.

3. Lexical Conventions (어휘 규칙)

Verilog의 어휘 규칙은 C 언어와 매우 유사하다.

식별자(Identifier): 영숫자, *, $로 구성. 대소문자 구분(case-sensitive). 첫 글자는 문자,* , 또는 $ (숫자로 시작 불가). 예: counter, four_bit_adder, _4b_adder

키워드(Keywords): 예약어. and, assign, begin, case, else, end, module, wire, reg, always 등. 모두 소문자.

주석(Comments): // 한 줄 주석, /* */ 여러 줄 주석.

숫자 표현 — 매우 중요!

형식: `<size>'<base>value`

- size: 비트 수. 생략 시 32비트로 가정
- ': 작은따옴표 (backtick ` 아님!)
- base: b/B (2진), o/O (8진), d/D (10진), h/H (16진). 생략 시 10진
- value: 상수 값

크기 불일치 시 처리 규칙:

```
크기 < 값:  MSB가 잘린다 (truncation)
            예: 4'b11110010 → 4'b0010 (상위 4비트 잘림)

크기 > 값:  왼쪽을 채운다 (padding)
            - 최상위가 0 또는 1이면 → 0으로 채움
            - 최상위가 x이면 → x로 채움
            - 최상위가 z이면 → z로 채움
```

숫자 표현 예제:

```
 표현식                 저장되는 값
──────────────────────────────────────────────────────────
 1                    00000000000000000000000000000001  (32비트
decimal)
 8'hAA                10101010                          (8비트)
 6'b10_0011           100011                            (6비트)
 'hF                  00000000000000000000000000001111  (32비트)
 6'hCA                001010                            (6비트,
CA=11001010 → 하위 6비트만)
 16'bZ                zzzzzzzzzzzzzzzz                  (16비트 high-
Z)
 8'bx                 xxxxxxxx                          (8비트
unknown)
```

음수: -'value. 2의 보수(2's complement) 형식.

```
a = 14'h1234       → 00001234
a = -14'h1234      → ffffedcc  (32비트 reg에서 2의 보수)
a = 32'hDEAD_BEEF  → deadbeef
a = -32'hDEAD_BEEF → 21524111  (2의 보수)
```

특수 문자: _(underscore)는 가독성 용도. ?(물음표)는 z와 동일. Four-value logic: 0, 1, x(unknown), z(high-impedance).

4. Data Types — wire vs reg

이것이 Verilog에서 가장 흔한 혼동 포인트이다.

Nets (연결선): wire, tri, wand 등. 하드웨어 연결점(connection point). 값을 저장할 수 없다! primitive, continuous assignment, module port에 의해 구동됨. Verilog의 wire는 방향성이 있다(directional).

Variables (변수): reg, integer, real, time, realtime. 데이터 저장 소자를 표현. procedural statement, task, function 내에서만 값 할당 가능. input이나 inout 포트가 될 수 없다.

함정 주의: reg가 반드시 물리적 레지스터(flip-flop)를 의미하는 것이 아니다! reg는 단지 "procedural block에서 값을 할당받을 수 있는 변수"라는 의미이다. 합성 시 조합 논리로 구현될 수도 있고, 레지스터로 구현될 수도 있다.

wire와 reg의 사용 규칙:

```
wire: continuous assignment (assign)의 좌변(LHS), module의
```

```
input/inout port
      구동 원(driver)이 있어야 함

reg:  procedural block (always/initial)의 좌변(LHS)
      output port로 사용 가능 (output reg)
      구동 원 없이 값을 유지할 수 있음
```

5. Verilog Assignments — 세 가지 할당 방식

1. Continuous Assignment (연속 할당): net(wire)에 값을 구동(drive). 조합 논리를 모델링. assign 키워드 사용. 항상 구동 상태이다. RHS의 값이 바뀔 때마다 LHS가 즉시 업데이트된다.

```
// 명시적 (explicit)
wire temp;
assign temp = a | b;

// 암시적 (implicit) — 선언과 할당 결합
wire temp = a | b;
```

2. Procedural Assignment (절차적 할당): reg 타입 변수에 always 블록/task/function 내에서 값 할당. 보통 레지스터와 FSM을 모델링.

3. Procedural Continuous Assignment (절차적 연속 할당): assign/deassign으로 변수에 대한 모든 절차적 할당을 override. 특수한 경우에 사용 (일반적으로 잘 쓰지 않음).

6. Verilog Ports (포트)

포트는 모듈을 외부 세계와 연결하는 창구이다.

```
  ┌──────────────────────────────────────┐
  │              Module                  │
net/var ─→■ input port ─→ net            │
  │           net ←─ output port ■←─ net/var
net ─→■←─ inout port ─→←─ net            │
  └──────────────────────────────────────┘
```

기본값: 포트는 기본적으로 wire 타입, unsigned. output 포트만 reg 타입으로 선언 가능(output reg).

Port Connection 두 가지 방식:

```
// 순서에 의한 연결 (positional) — 비추천
half_adder ha_1 (x, y, s1, c1);

// 이름에 의한 연결 (named) — 추천!
half_adder ha_2 (.x(cin), .y(s1), .s(s), .c(c2));
```

이름에 의한 연결을 사용하라! 순서가 무관하고, 포트가 많을 때 실수를 줄일 수 있다.

Unconnected/Floating ports: input이 연결되지 않으면 z로 구동, output이 연결되지 않으면 floating.

주의: built-in primitive 게이트(and, or, xor 등)는 instance name을 생략할 수 있지만, 사용자 정의 모듈을 인스턴스화할 때는 instance name이 필수이다.

7. Port Connection 전체 예제

```
module half_adder (input  x, input  y,
                   output s, output c);
  xor (s, x, y);    // Verilog built-in primitive
  and (c, x, y);
endmodule
```

합성 회로도 (Half Adder):

```
x ──→┤ XOR ├──→ s    (S = X ⊕ Y)
y ──→┤     │
     └─────┘

x ──→┤ AND ├──→ c    (C = X · Y)
y ──→┤     │
     └─────┘
```

```
module full_adder (input  x, input  y, input  cin,
                   output s, output cout);
  wire s1, c1, c2;
  half_adder ha_1 (x, y, s1, c1);                    //
positional
  half_adder ha_2 (.x(cin), .y(s1), .s(s), .c(c2));  // named
  or (cout, c1, c2);
endmodule
```

합성 회로도 (Full Adder):

```
x ──→┤ Half   ├─s1──→┤ Half   ├──→ sum
y ──→┤ Adder  │       │ Adder  │
     │ ha_1   ├─c1─┐  │ ha_2   ├─c2─┐
     └────────┘    │  └────────┘    │
            cin────┘                │
                  └──→┤ OR ├──→ cout ←┘
```

8. Module Modeling Levels — 세 가지 모델링 레벨

(1) Structural Modeling (구조적 모델링)

게이트 레벨에서 회로를 기술. Built-in primitive, 사용자 정의 primitive, 또는 다른 모듈을 wire로 연결. 계층 구조(hierarchy)를 기술.

(2) Dataflow Modeling (데이터플로우 모델링)

입력에서 출력으로의 데이터 흐름을 기술. 연산자와 continuous assignment(assign) 사용. 조합 논리에 적합.

```
module full_adder_dataflow(input x, input y, input c_in,
                           output sum, output c_out);
  assign #5 {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도 (Dataflow Full Adder):

```
x ──→┤        ├──→ sum     {c_out, sum} = 5비트 결과
y ──→┤  ADD   │             MSB = c_out, 하위4비트 = sum
c_in→┤        ├──→ c_out
     └────────┘
```

{c_out, sum}은 concatenation 연산으로, c_out과 sum을 하나의 2비트 결과로 묶은 것이다.

(3) Behavioral Modeling (행위적 모델링)

always 블록과 procedural statement 사용. 대상(LHS)은 반드시 reg 타입.
순차 논리와 조합 논리 모두 가능.

```
module full_adder_behavioral(input x, input y, input c_in,
                             output reg sum, output reg c_out);
  always @(x, y, c_in)
    #5 {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도: Dataflow와 동일한 하드웨어로 합성됨 (조합 논리, 메모리 소자 없음). reg 타입이지만 always @(*)이므로 조합 논리.

always @(x, y, c_in): sensitivity list에 있는 신호 중 하나라도 값이 바뀌면 always 블록이 활성화된다. always @(*)로 쓰면 블록 내에서 읽히는 모든 신호에 자동으로 민감하게 된다(Verilog-2001 이후, 조합 논리에서 추천).

(4) Mixed-Style Modeling

세 가지 모델링 레벨을 하나의 모듈 안에서 섞어 사용. 대규모 설계에서 흔히 사용.

```
module full_adder_mixed_style(input x, input y, input c_in,
                              output sum, output reg c_out);
  wire s1, c1, c2;

  // Structural: Half Adder 1
  xor xor_ha1(s1, x, y);
  and and_ha1(c1, x, y);

  // Dataflow: Half Adder 2
  assign sum = c_in ^ s1;
  assign c2  = c_in & s1;

  // Behavioral: Output OR gate
  always @(c1, c2)
    c_out = c1 | c2;
endmodule
```

합성 회로도 (Mixed — 동일 하드웨어):

```
x──→┤XOR├─s1─→┤XOR├─→ sum     (HA1: structural)
y──→┤   |      |   |           (HA2: dataflow)
    └───┘  cin→┤   |
               └───┘

x──→┤AND├─c1─→┤OR├─→ c_out   (OR: behavioral)
y──→┤   |      |  |
    └───┘      |  |
      cin→┤AND├─c2→┤  |
      s1─→┤   |    └──┘
          └───┘
```

세 가지가 모두 동시에(concurrently) 동작한다.

RTL (Register-Transfer Level) = synthesizable behavioral + dataflow.
업계에서 가장 많이 사용하는 추상화 수준이다.

9. Verilog Simulation

설계한 모듈을 검증하기 위해 별도의 테스트벤치(testbench)를 사용한다.

```
┌──────────────────────────────────────┐
│          Top-Level Block             │
│  ┌──────────────┐   ┌──────────────┐ │
│  │  Stimulus    │──→│ Module Under │ │
│  │  Block       │←──│ Test (MUT)   │ │
│  │ (Testbench)  │   └──────────────┘ │
│  └──────────────┘                    │
└──────────────────────────────────────┘
```

System Tasks:

```
$display(ep1, ep2, ..., epn);  // 값 출력 (한 번)
$monitor(ep1, ep2, ..., epn);  // 값 변화 시마다 자동 출력
$monitoron;   $monitoroff;     // 모니터링 활성화/비활성화
$stop;                          // 시뮬레이션 일시 정지
$finish;                        // 시뮬레이션 종료
```

$display와 $monitor의 차이: $display는 호출 시점에 한 번만 출력하지만, $monitor는 인자의 값이 변할 때마다 자동으로 출력한다.

Timescale: `timescale time_unit / time_precision`. 예:timescale 1ns / 1ps → #15는 15ns. FPGA 설계에서는 ns를 time unit으로 권장.

테스트벤치 예제 — Clock Generation:

```
module tb;
  reg clk;
  always #5 clk = ~clk;    // 5ns마다 반전 → 주기 10ns 클럭
  initial begin
    $monitor("Time = %0t clk = %0d", $time, clk);
    #5  clk = 0;    // 5ns에 clk을 0으로
    #80 $finish;    // 85ns에 시뮬레이션 종료
  end
endmodule
```

4:2 Encoder 테스트벤치:

```
`timescale 1ns / 100ps
module encoder4to2_tb;
  reg  [3:0] din;
  wire [1:0] dout;
  encoder4to2 encoder(
    .Y0(din[0]), .Y1(din[1]), .Y2(din[2]), .Y3(din[3]), .A(dout));
  initial begin
    $monitor("Time=%0t din=%0b dout=%0b", $realtime, din, dout);
    #5    din = 4'b1000;   // 5ns
    #10   din = 4'b0100;   // 15ns
    #20   din = 4'b0010;   // 35ns
    #15   din = 4'b0001;   // 50ns
    #1    din = 4'b1000;   // 51ns
    #0.5  din = 4'b0100;   // 51.5ns
    #0.15 din = 4'b0010;   // 51.65→51.7ns (100ps 해상도 반올림)
    #10   $finish;          // 61.7ns
  end
endmodule
```

시뮬레이션 결과:

```
Time = 0       din = xxxx  dout = xx
Time = 5000    din = 1000  dout = 11   (Y3=1 → A=3)
Time = 15000   din = 100   dout = 10   (Y2=1 → A=2)
Time = 35000   din = 10    dout = 1    (Y1=1 → A=1)
```

```
Time = 50000   din = 1     dout = 0    (Y0=1 → A=0)
Time = 51000   din = 1000  dout = 11
Time = 51500   din = 100   dout = 10
Time = 51700   din = 10    dout = 1    (51.65→51.7 반올림)
```

Operator Precedence (높→낮): ~ > *,/,% > +,- > <<,>>,<<<,>>> > <,<=,>,>= > ==,!=,===,!== > &,~& > ^~ > |,~| > && > || > ?:

Ch3: Dataflow Modeling

이 챕터에서는 Verilog의 데이터플로우 모델링을 깊이 다룬다. 핵심은 두 가지: (1) Continuous Assignment의 상세 규칙과 지연 모델, (2) Verilog의 모든 연산자 종류와 동작 특성(특히 signed/unsigned 비교, x/z 처리, shift 연산).

1. Structural Modeling의 한계와 Dataflow Modeling의 동기

Structural modeling(게이트 인스턴스화)은 복잡한 회로에서는 비현실적이다. 수백 개의 게이트를 일일이 연결하는 것은 유지보수가 불가능하다. Dataflow modeling은 게이트 프리미티브 대신 연산자를 사용하여 입력에서 출력으로의 데이터 흐름을 기술한다.

```
// Structural (불편)              // Dataflow (간결)
xor (s, x, y);                    assign sum   = x ^ y;
and (c, x, y);                    assign c_out = x & y;
```

2. Continuous Assignments — 상세 규칙

Continuous assignment는 dataflow modeling의 가장 기본 문장이다.

문법: assign [drive_strength] [delay] = ;

핵심 특성: - net(wire)에만 값을 구동 (variable/reg에는 사용 불가!) - vector와 scalar 모두 가능 - Always Active: RHS의 값이 변할 때마다 즉시 LHS가 갱신됨 - 조합 논리를 논리 표현식으로 모델링

선언과 할당 방식 3가지:

```
// (1) Regular: 별도 선언 + assign (추천)
wire out;
assign out = in1 & in2;

// (2) Implicit continuous assignment: 선언과 동시에 할당
wire out = in1 & in2;

// (3) Implicit net declaration: wire 선언 없이 assign (비추천!)
assign out = in1 & in2;   // out이 wire로 암시 선언됨
```

중요 규칙: - net은 한 번만 선언 가능. `wire out = in1 & in2;` 뒤에 `wire out = in3;`은 불법 - 하나의 net에 여러 continuous assignment 가능 (하지만 특별한 이유 없으면 하지 마라 — 멀티 드라이버) - Implicit net declaration은 비추천: 원했던 게 reg인데 실수로 wire가 되는 버그 유발 - LHS는 반드시 scalar/vector net 또는 net의 concatenation (절대 reg가 아님!) - RHS에는 scalar/vector reg, function call 등 사용 가능

3. Delays (지연)

지연은 RHS 값이 변한 시점부터 LHS에 할당되기까지의 시간이다.

```
// (1) Regular continuous assignment delay — inertial delay
assign #10 out = in1 & in2;

// (2) Implicit continuous assignment delay — inertial delay
wire #10 out = in1 & in2;

// (3) Net delay — transport delay
wire #10 out;
assign out = in1 & in2;
```

Inertial delay (관성 지연): 게이트의 RC를 모델링. continuous assignment에서 기본 사용. 지연보다 짧은 펄스는 무시(걸러짐).

Transport delay (전파 지연): 와이어의 지연을 모델링. net delay에서 기본 사용. 모든 변화가 그대로 전파됨.

합성 시 #delay는 무시된다. 시뮬레이션 전용.

4. Constants — signed/unsigned 상세 (시험 핵심 함정)

단순 10진수(-123, 12345)는 signed로 취급. Base format 표기(16'habcd)는 기본 unsigned이며, 's'를 쓰면 signed.

핵심 원칙 두 가지: - 원칙 1: 음수는 2의 보수로 표현된다. - 원칙 2: 's' 지정자는 비트 패턴을 바꾸지 않는다. 해석(interpretation)만 바꾼다.

상세 예제 (시험 필출):

```
4'sb1001  → 비트: 1001  → signed 해석 → -7
5'sb1001  → 비트: 01001 → signed 해석 → +9 (크기 큼 → 0 패딩)
-4'sb1001 → -(-7) = +7
4'shf     → 비트: 1111  → signed 해석 → -1
4'hf      → 비트: 1111  → unsigned 해석 → 15
5'shf     → 비트: 01111 → signed 해석 → +15
4'sd12    → 비트: 1100  → signed 해석 → -4  (12=1100)
5'sd12    → 비트: 01100 → signed 해석 → +12
3'sd12    → 비트: 100   → signed 해석 → -4  (truncation)
-4'sd12   → -(-4) = +4
-4'sb0010 → -(2) → 비트패턴: 1110
4'd -12   → 불법! (마이너스는 맨 앞에만)
```

5. Variable Data Types — 상세

reg: 할당 사이에 값을 유지. 기본 unsigned. signed 선언 가능. 초기값: x.

```
reg a, b;                    // 1-bit reg
reg [7:0] data_a;            // 8-bit, MSB = bit 7
reg signed [7:0] d;          // 8-bit signed reg
```

integer: signed reg로 취급, LSB at bit 0. 최소 32비트. 초기값: x. time: unsigned, 최소 64비트. $time과 함께 시뮬레이션 시간 저장용. 초기값: x. real / realtime: 서로 동일, 교환 가능. range 선언 불가. 초기값: 0.0 (reg/integer/time과 다름!).

6. Vectors, Bit-Select, Part-Select

Bit endianness가 중요하다:

```
reg [7:0] a = 14;    // MSB=bit7, a=00001110, a[3:0]=1110 ✓
reg [0:7] b = 14;    // MSB=bit0, b=00001110, b[0:3]=0000 ✓
                     // b[3:0]은 불법! (endianness 뒤집기 불가)
```

Variable part-select는 루프에서 벡터를 슬라이싱할 때 핵심적으로 사용된다:

```
wire [15:0] big_vect;
big_vect[8+:8]   // = big_vect[15:8]  (8에서 위로 8비트)
big_vect[7-:8]   // = big_vect[7:0]   (7에서 아래로 8비트)
// width는 반드시 상수, base는 변수 가능
```

7. Array and Memory

한 번에 하나의 배열 요소만 접근(할당) 가능:

```
reg [7:0] mema [7:0];           // 8개의 8-bit word
reg [7:0] memb [3:0][3:0];     // 4x4의 8-bit word

mema[4][3]          // 합법: 4번째 word의 3번 비트
mema[5][7:4]        // 합법: 5번째 word의 상위 4비트
mema = 0;           // 불법! 전체 메모리 한 번에 할당 불가
memb[1] = 0;        // 불법! 행 전체 할당 불가
mema[1] = 0;        // 합법 (단일 word)
memb[1][3] = 0;     // 합법 (단일 word)
```

8. Operators — Verilog 연산자 총정리

Arithmetic Operators

+, -, *, /, **, %. 피연산자에 x 또는 z 비트가 있으면 결과는 전부 x. 나눗셈/나머지에서 2번째 피연산자가 0이면 결과는 x. %(modulus)는 합성 가능하지만 가능하면 피하라(하드웨어 비용 높음).

Relational Operators — signed/unsigned 함정!

<, >, <=, >=. 결과: true→1, false→0, x/z 있으면→x.

규칙: 피연산자 중 하나라도 unsigned이면, 둘 다 unsigned로 비교. 둘 다 signed이면 sign-extension 후 signed 비교.

```
wire signed [3:0] a = -4'd4;    // a = 1100 (signed → -4)
wire unsigned [3:0] b = 4'd5;   // b = 0101 (unsigned → 5)

a > b → ?
// b가 unsigned → 둘 다 unsigned로 비교
// a = 1100 = 12(unsigned), b = 0101 = 5
// 12 > 5 → 결과: 1 (true!)
// signed로 보면 -4 > 5는 false인데, unsigned 비교라서 true!
```

Equality Operators

== (logical equality): x/z 비트가 있으면 결과가 x. === (case equality): x와 z까지 정확히 비교. 결과는 항상 0 또는 1. 합성 불가.

```
4'b1x0z === 4'b1x0z   // → 1 (정확히 일치)
4'b1x0z == 4'b1x0z    // → x (x/z 비트 있으므로)
```

Equality with sign extension:

```
wire signed [5:0] a = -6'd4;   // a = 111100
wire signed [3:0] b = -4'd4;   // b = 1100 → sign ext → 111100
a == b → 1 (true)
```

Concatenation & Replication Operators

```
a = 1'b1, b = 2'b00, c = 3'b101
{a, b[0], c[2:1]}  → 4'b1010
{ {1{a}}, {2{b[0]}}, {3{c[2:1]}} }  → 9'b1_00_10_10_10
```

피연산자는 반드시 sized여야 한다.

응용 예제: 4-bit Adder

```
module four_bit_adder(input [3:0] x, input [3:0] y, input c_in,
                      output [3:0] sum, output c_out);
  assign {c_out, sum} = x + y + c_in;
endmodule
```

합성 회로도:

```
x[3:0] ──→┤        ├──→ sum[3:0]
y[3:0] ──→┤  ADD   │
c_in   ──→┤        ├──→ c_out (MSB = carry)
          └────────┘
```

응용 예제: 4-bit Adder/Subtractor — 핵심 트릭!

```
module twos_adder(input [3:0] x, input [3:0] y, input c_in,
                  output [3:0] sum, output c_out);
  wire [3:0] t;
  assign t = y ^ {4{c_in}};            // c_in=0→t=y, c_in=1→t=~y
  assign {c_out, sum} = x + t + c_in;  // c_in=1이면 x+~y+1 = x-y
endmodule
```

합성 회로도:

```
y[3:0]──→┤ XOR ├──→ t[3:0]──→┤        ├──→ sum[3:0]
c_in ──→┤{4{}}├              │  ADD   │
         └──────┘   x[3:0]──→┤        ├──→ c_out
                    c_in ──→┤        │
                            └────────┘

c_in=0: t=y^0000=y     → x+y+0 = x+y   (덧셈)
c_in=1: t=y^1111=~y    → x+~y+1 = x-y  (뺄셈, 2의 보수)
```

이것이 하나의 가산기로 덧셈과 뺄셈을 모두 수행하는 핵심 트릭이다.

Conditional Operator

```
assign out = selection ? in_1 : in_0;                  // 2:1 Mux
assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);   // 4:1 Mux
```

합성 회로도 (4:1 Mux):

```
i0──→┤MUX├──→─┐
i1──→┤2:1│     │  ┌──→┤MUX├──→ out
     └───┘  s0 └──┘   │2:1│
i2──→┤MUX├──→──────────┤   │
i3──→┤2:1│      s1──→┤   │
     └───┘  s0        └───┘
```

condition이 x이면 양쪽 모두 평가하여 비트별 비교. 같으면 그 값, 다르면 x.

Reduction Operators — 벡터를 스칼라로

```
assign ep   = ^x;      // even parity: 모든 비트 XOR
assign op   = ~ep;      // odd parity
assign zero = ~(|x);   // all-zero: OR reduction → NOT
assign one  = &x;      // all-one: AND reduction
```

합성 회로도 (Parity Generator):

```
x[0]──→┤XOR├──→┤XOR├──→┤XOR├──→...──→┤XOR├──→ ep
x[1]──→┤    │   │    │   │    │       │    │
       └────┘   └────┘   └────┘       └────┘
x[2]──→┘ x[3]┘ x[4]┘   ...  x[8]┘
ep──→┤NOT├──→ op
```

합성 회로도 (All-Zero / All-One):

```
x[0]──→┤OR├──→┤OR├──→...──→┤NOT├──→ zero
x[1]──→┤  │   │  │
...     └──┘   └──┘

x[0]──→┤AND├──→┤AND├──→...──→ one
x[1]──→┤    │   │    │
...     └────┘   └────┘
```

Logical Operators

!, &&, ||. Bitwise와의 차이: Logical은 전체 피연산자를 true/false로 평가하여 1비트 결과. Bitwise는 비트별 연산.

```
reg [7:0] c = 123, d = 0;
a = c || d;    // a = 1 (123은 true, 0은 false, true OR false = true)
b = c && d;    // b = 0 (true AND false = false)
```

Bitwise Operators

~, &, |, ^, ~^. z는 bitwise에서 x로 취급된다. 짧은 피연산자는 zero-extend 후 연산.

x/z 비트 처리 진리표:

```
&  | 0  1  x  z        |  | 0  1  x  z
---+----------------   ---+----------------
0  | 0  0  0  0        0  | 0  1  x  x
1  | 0  1  x  x        1  | 1  1  1  1
x  | 0  x  x  x        x  | x  1  x  x
z  | 0  x  x  x        z  | x  1  x  x
```

Shift Operators

논리 시프트(<<, >>): 빈 자리를 항상 0으로 채움. 산술 시프트(<<<, >>>): 좌시프트는 빈 자리 0(논리와 동일, 부호 비보존!). 우시프트는 부호 비트(sign bit)로 채움.

```
8비트 signed 값 = 10101010 (-86)

Logical >>1:     01010101            (0으로 채움)
Arithmetic >>>1: 11010101 (-43)      (부호비트 1로 채움)

Logical <<1:     01010100            (0으로 채움)
```

```
Arithmetic <<<1: 01010100 (84)       (0으로 채움, 부호 변경됨!)
```

함정: 산술 좌시프트는 논리 좌시프트와 동일하게 동작한다. 부호가 보존되지 않을 수 있다! 산술 우시프트만 부호 보존 효과가 있다.

Ch4: Behavioral Modeling

이 챕터는 Verilog의 행위적 모델링(Behavioral Modeling)을 다룬다. 이전 챕터의 Dataflow modeling이 assign과 연산자로 조합 논리를 기술했다면, Behavioral modeling은 always/initial 블록 안에서 절차적(procedural) 문장을 사용하여 순차 논리와 조합 논리 모두를 기술한다. 이 챕터에서 가장 중요한 것은 단연 Blocking vs Nonblocking Assignment의 차이이다.

Part I: Procedural Blocks & Assignments

1. initial Block

initial 블록은 변수를 초기화하거나 특정 값을 구동하는 데 사용한다. 시뮬레이션 동안 딱 한 번만 실행되며, 여러 initial 블록은 time 0에서 병렬로 시작된다.

```
initial statement;
initial begin statement1; statement2; end

reg clk;
initial clk = 0;        // 일반적
reg clk = 0;             // 결합형 (동일 효과)

module inc(input [3:0] x,
           output reg [3:0] y = 4'b1000);  // 포트+초기화 결합
endmodule
```

initial은 합성 가능한가? ASIC에서는 합성 불가능하다. FPGA에서는 가능할 수 있지만 시간 지연은 합성 시 무시된다. 가이드라인: 설계 모듈에서는 initial을 사용하지 마라. 테스트벤치에서만 사용하라.

2. always Block — 순차/조합 논리와 회로도

always 블록은 sensitivity list의 이벤트가 발생할 때마다 실행된다. sensitivity list와 내부 문장에 따라 순차 논리(flip-flop, latch)와 조합 논리를 모두 만들 수 있다.

D Flip-Flop:

```
module flop(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(posedge CLK)
    Q <= D;    // nonblocking assignment
endmodule
```

합성 회로도:

```
D[3:0]──→┤ D   Q ├──→ Q[3:0]
         │  DFF  │
CLK ──→──┤>      │         4개의 DFF (4-bit register)
         └───────┘
```

posedge: CLK의 양(positive) 에지(0→1)에서 실행. negedge: 음(negative) 에지(1→0)에서 실행.

D Latch:

```
module my_latch(input CLK, input [3:0] D, output reg [3:0] Q);
  always @(CLK, D)
    if (CLK == 1) Q <= D;
endmodule
```

합성 회로도:

```
D[3:0]──→┤ D   Q ├──→ Q[3:0]
         │ LATCH │
CLK ──→──┤ EN    │        4개의 D Latch (CLK=1일 때 transparent)
         └───────┘
```

Combinational — Inverter:

```
module inverter(input [3:0] a, output reg [3:0] y);
  always @(*)
    y = ~a;    // blocking assignment
endmodule
```

합성 회로도:

```
a[3:0]──→┤ NOT ├──→ y[3:0]
         └─────┘

→ 인버터만! 메모리 소자 없음 (reg 타입이지만 FF/Latch 아님)
```

always @(): *블록 내에서 읽히는 모든 신호에 자동 민감. always @(a)와 동일하지만, 신호를 빠뜨릴 위험이 없으므로* @()가 더 좋은 스타일이다.

합성 결과: 메모리 소자 없음! reg ≠ 물리 레지스터 다시 확인.

3. Unwanted Latch — 조합 논리의 가장 흔한 함정!

조합 논리를 always 블록으로 기술할 때, if문에 else가 없으면 의도치 않은 latch가 생성된다.

```
// 나쁜 예: else 없음 → unwanted latch!
always @* begin
  if (Y0 == 1 && ...) A = 2'b00;
  else if (Y1 == 1 && ...) A = 2'b01;
  // else가 없다! → 위 조건에 모두 해당하지 않으면 A 유지 → latch!
end

// 올바른 예: else로 모든 경우를 커버
always @* begin
  if (Y0 == 1 && ...) A = 2'b00;
  else if (Y1 == 1 && ...) A = 2'b01;
  else A = 2'b00;    // default → latch 방지!
end
```

합성 회로도 비교:

```
[else 없을 때 — Latch 합성됨!]
조건──→┤ Comb  ├──→┤ LATCH ├──→ A
       │ Logic │   │  EN   │
       └───────┘   └───────┘

       ★ 의도치 않은 메모리 소자!

[else 있을 때 — 순수 조합 논리]
조건──→┤ Comb  ├──→ A
       │ Logic │
```

```
└──────┘
★ Latch 없음!
```

원칙: 조합 논리 always 블록에서는 if에 반드시 else를, case에 반드시 default를 넣어서 모든 경우를 커버하라.

4. Blocking vs Nonblocking Assignment — 핵심!

Blocking Assignment (=): 순서대로 실행. 현재 문장의 할당이 완료될 때까지 다음 문장이 차단됨.

```
initial begin
  x = #5 1'b0;    // x는 time 5에 0
  y = #3 1'b1;    // y는 time 8에 1  (5+3, 순차적!)
  z = #6 1'b0;    // z는 time 14에 0 (8+6)
end
```

Nonblocking Assignment (<=): 모든 RHS를 동시에 평가하고, 할당은 블록 끝에서 한꺼번에.

```
initial begin
  x <= #5 1'b0;   // x는 time 5에 0
  y <= #3 1'b1;   // y는 time 3에 1  (모두 time 0에서 동시 평가!)
  z <= #6 1'b0;   // z는 time 6에 0
end
```

Blocking vs Nonblocking — Full Adder 비교:

a가 0→1, b=0, cin=0인 상태에서:

Blocking: p=a^b=1(즉시) → g=a&b=0(즉시) → s=p$^{cin=1}$0=1(p의 새 값!) → cout=0. 한 번에 올바른 결과 (p=1,g=0,s=1,cout=0).

Nonblocking: 1차 — p←1,g←0,s←0^0=0(p의 이전 값!),cout←0. p가 변했으므로 재실행. 2차 — s←1^0=1. 최종 같은 결과지만 2번 실행! 시뮬레이션 느림 + sensitivity list 빠뜨리면 틀린 결과.

결론: 조합 논리에서는 Nonblocking을 사용하지 마라!

5. Shift Register — Blocking vs Nonblocking의 결정적 차이

```
// Blocking (잘못! FF 1개만 합성)
always @(posedge clk) begin
  qout[0] = sin;       // qout[0]이 sin으로 즉시 변경
  qout[1] = qout[0];   // qout[0]의 새 값(=sin) 사용!
  qout[2] = qout[1];   // 역시 sin!
  qout[3] = qout[2];   // 모두 sin! → FF 1개만 합성!
end
```

합성 결과 (잘못됨):

```
sin──→┤ DFF ├──→ qout[0] ═══ qout[1] ═══ qout[2] ═══ qout[3]
      └─────┘                (단순 wire 연결, FF 없음!)
       clk
→ 모든 비트가 sin. shift register가 아님!
```

```
// Nonblocking (올바름! FF 4개 체인)
always @(posedge clk) begin
  qout[0] <= sin;       // 모든 RHS를 이전 값으로 동시 평가
  qout[1] <= qout[0];   // qout[0]의 이전 값 사용!
  qout[2] <= qout[1];
```

```
        qout[3] <= qout[2];
      end
      // 더 좋은 표현: qout <= {qout[2:0], sin};
```

합성 결과 (올바름):

```
sin──→┤DFF├──→qout[0]──→┤DFF├──→qout[1]──→┤DFF├──→qout[2]──→┤DFF├──→(
        └───┘               └───┘               └───┘
└───┘
                                        clk (공유)
  → 4개의 DFF 체인. 올바른 shift register!
```

6. Race Condition

```
      // Blocking → race! (실행 순서에 따라 결과 다름)
      always @(posedge clock) x = y;
      always @(posedge clock) y = x;

      // Nonblocking → 올바른 swap (RHS 동시 평가)
      always @(posedge clock) x <= y;
      always @(posedge clock) y <= x;
```

Nonblocking의 3단계: (1) 모든 RHS 읽기 (2) RHS 평가→임시 저장 (3) 임시→LHS 할당.

7. count와 finish 예제

count=1, finish=0에서 posedge clk:

```
      // Blocking:
      count = count - 1;         // count 즉시 0
      if (count == 0) finish = 1; // count=0이므로 finish=1
      // 결과: finish = 1

      // Nonblocking:
      count <= count - 1;         // RHS: 1-1=0 (아직 할당 안 됨)
      if (count == 0) finish <= 1; // count의 현재값=1 → 조건 false
      // 결과: finish = 0
```

8. 5가지 가이드라인 — 필수 암기!

1. 순차 논리(sequential): always @(posedge clk) + nonblocking (<=)
2. 간단한 조합 논리: assign (continuous assignment, dataflow)
3. 복잡한 조합 논리: always @(*) + blocking (=)
4. 같은 신호를 여러 always/assign에서 할당하지 마라
5. 같은 always/initial 블록에서 blocking과 nonblocking을 섞지 마라

Part II: Timing Control, Selection, Iteration

9. Timing Controls

Inter-assignment delay: `#10 a = b;` — 문장 전체를 10단위 후 실행. Intra-assignment delay: `a = #10 b;` — RHS를 현재 시점에 평가, 할당만 10단위 후.

```
      // Blocking intra-assignment
```

```
initial begin
  y = #25 ~x;              // time 0에 ~x 평가, time 25에 y에 할당
  count = #15 count + 1;   // time 25에 count+1 평가, time 40에 할당
end

// Nonblocking intra-assignment
initial begin
  y <= #25 ~x;             // time 0에 ~x 평가, time 25에 y에 할당
  count <= #15 count + 1; // time 0에 count+1 평가, time 15에 할당!
end
```

Event Control: posedge(0→1방향), negedge(1→0방향). @(posedge clk_a, posedge clk_b, trig)로 여러 이벤트 결합. @(*)로 모든 읽히는 신호에 자동 민감.

Named Events: event declared → trigger(->) → recognized(@). 클럭 도메인 간 핸드셰이킹.

Level-sensitive: wait(!enable) #10 a = b; — enable이 0이 될 때까지 대기.

10. if...else & Mux 예제

```
module mux4_to_1_ifelse(input i0, i1, i2, i3,
                        input s1, s0, output reg out);
  always @(*)
    if (s1) begin
      if (s0) out = i3; else out = i2;
    end
    else begin
      if (s0) out = i1; else out = i0;
    end
endmodule
```

합성 회로도:

```
i0──→┤MUX├──→┐
i1──→┤2:1├   │   ┌──→┤MUX├──→ out
     └───┘s0 └───┘   │2:1│
i2──→┤MUX├──→────────┤   │
i3──→┤2:1├    s1──→┤   │
     └───┘s0         └───┘
```

11. Counter with 비동기 리셋

```
module counter(input clock, input clear, output reg [3:0] qout);
  always @(negedge clock or posedge clear) begin
    if (clear) qout <= 4'd0;       // 비동기 리셋 (우선)
    else       qout <= (qout + 1); // negedge에서 카운트
  end
endmodule
```

합성 회로도:

```
                ┌─────┐        ┌─────┐
qout[3:0]──→┤  +  ├──→┤ 0 │MUX├──→┤ D   Q ├──→ qout[3:0]
         1──→┤     │   │   │   │   │ DFF   │
                └─────┘   │   │   │   │  R    │
         4'd0──────────→┤ 1 │   │   └─────┘
                          └───┘   │    negedge clock
                clear──→──sel     │    posedge clear (비동기)
```

negedge clock or posedge clear: clear가 posedge로 올라가면 클럭과 무관하게 즉시 리셋(비동기 리셋).

12. case / casex / casez

```
module mux_4x1_case(input I0,I1,I2,I3, input [1:0] S, output reg Y);
  always @(*)
    case (S)
      2'b00: Y=I0; 2'b01: Y=I1; 2'b10: Y=I2; 2'b11: Y=I3;
    endcase
endmodule
```

합성 회로도:

```
I0──→┤        ├
I1──→┤  4:1   ├──→ Y
I2──→┤  MUX   │
I3──→┤        │
     └────────┘
S[1:0]──→┤sel│
```

casex (x와 z를 don't care):

```
module trailing_zero_4b(input [3:0] data, output reg [2:0] out);
  always @(data)
    casex (data)
      4'bxxx1: out = 0;  4'bxx10: out = 1;
      4'bx100: out = 2;  4'b1000: out = 3;
      4'b0000: out = 4;  default: out = 3'b111;
    endcase
endmodule
```

합성 회로도 (priority encoder):

```
data[0]──→┐
data[1]──→┤ Priority    ├──→ out[2:0]
data[2]──→┤ Encoder     │
data[3]──→┤             │
          └─────────────┘
```

13. Loop Constructs

루프는 initial 또는 always 블록 안에서만 사용 가능. 합성 시 unroll되어 동일 하드웨어 생성.

while:

```
module zero_count_while(input [7:0] data, output reg [3:0] out);
  integer i;
  always @(data) begin
    out = 0; i = 0;
    while (i <= 7) begin
      if (data[i] == 0) out = out + 1;
      i = i + 1;
    end
  end
endmodule
```

주의: blocking을 nonblocking으로 바꾸면 out과 i가 루프 내에서 업데이트되지 않아 무한 루프!

for:

```
module zero_count_for(input [7:0] data, output reg [3:0] out);
  integer i;
  always @(data) begin
    out = 0;
    for (i = 0; i <= 7; i = i + 1)
      if (data[i] == 0) out = out + 1;
  end
endmodule
```

합성 회로도 (for loop unroll):

```
data[0]──→┤==0?├─→┐
data[1]──→┤==0?├─→┤ 8개의 비교기
...                │ + 가산기 체인
data[7]──→┤==0?├─→┤──→ out[3:0]
            └────┘   │
                     (합성 시 for는 unroll되어
                      8개의 병렬 비교+누적 가산)
```

8-bit Left Shift Register (for loop):

```
module lshift_reg(input clk, rstn, input [7:0] load_val,
                  input load_en, output reg [7:0] op);
  integer i;
  always @(posedge clk) begin
    if (!rstn) op <= 0;
    else if (load_en) op <= load_val;
    else begin
      for (i = 0; i < 7; i = i + 1)
        op[i+1] <= op[i];
      op[0] <= op[7];
    end
  end
endmodule
```

합성 회로도:

```
op[7]──→┤DFF├──→op[0]     (circular shift)
op[0]──→┤DFF├──→op[1]
op[1]──→┤DFF├──→op[2]
...
op[6]──→┤DFF├──→op[7]
         + MUX (load_en) + async reset (rstn)
```

repeat: 고정 횟수 반복. counter_expr는 시작 전 1회만 평가.

forever: while(1)과 동치. $finish 또는 disable로만 종료. 반드시 timing control과 함께 사용.

```
initial begin
  clock <= 0;
  forever begin
    #10 clock <= 1;
    #5  clock <= 0;
  end
end
// 주기 15 (HIGH 10, LOW 5) 클럭 생성
```

Ch5: Structural Modeling

이 챕터는 Verilog의 구조적 모델링(Structural Modeling)을 깊이 다룬다. Ch2에서 간단히 소개했던 내용을 확장하여, 14가지 게이트 프리미티브의 상세, 다양한 net 타입(wire/tri, wand/wor, tri0/tri1), 게이트 지연의 세밀한 지정 방법(rise/fall/turn-off, min:typ:max), 그리고 회로 설계에서 반드시 알아야 하는 해저드(hazard) 문제와 그 해결법을 다룬다.

1. Structural Modeling이란?

Structural modeling은 더 단순한 모듈들과 그들 사이의 연결(interconnections)로 모듈을 기술하는 방법이다. Verilog는 미리 정의된 게이트와 스위치(primitives)를 제공하며, 이들은 모듈처럼 인스턴스화된다.

```
module half_adder(input x, input y,
                  output s, output c);
  xor my_xor(s, x, y);
  and my_and(c, x, y);
endmodule
```

Structural modeling 모듈이 포함할 수 있는 것들: 다른 모듈들(계층적 설계), Gate primitives(14종, 합성 가능), Switch primitives(12종, 트랜지스터 레벨, 일반적으로 합성 불가), User-defined primitives(UDPs, 합성기에서 보통 지원 안 함). 이들을 wire(net)로 연결한다.

2. Gate Primitives — 14종 상세

and/or/xor 게이트 (6종)

and, or, xor, nand, nor, xnor — 하나의 scalar 출력, 여러 scalar 입력.

인스턴스화 문법: gate_name [instance_name] (output, input1, input2, ..., inputn);

x/z 포함 시 진리표:

AND 진리표:

	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

OR 진리표:

	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

XOR 진리표:

	0	1	x	z
0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

핵심: AND에서 입력 하나가 0이면 결과는 반드시 0. OR에서 입력 하나가 1이면 결과는 반드시 1. XOR에서는 x/z가 하나라도 있으면 항상 x.

buf/not 게이트

하나의 scalar 입력, 여러 scalar 출력. 문법: gate_name [instance_name] (output1, ..., outputn, input);

buf: 0→0, 1→1, x→x, z→x. not: 0→1, 1→0, x→x, z→x. z 입력은 x로 변환.

bufif/notif 게이트 (Tri-state 버퍼, 4종)

3-state 출력(0, 1, z). 문법: gate_name [instance_name] (output, input, control);

- bufif0: control=0이면 활성(입력 통과), control=1이면 z (active-low)
- bufif1: control=1이면 활성(입력 통과), control=0이면 z (active-high)
- notif0/notif1: bufif + 반전

Tri-state 버퍼를 이용한 2:1 Mux:

```
module two_to_one_mux_tristate(input x, y, s, output f);
  tri f;
  bufif0 b1 (f, x, s);   // s=0이면 x 통과
  bufif1 b2 (f, y, s);   // s=1이면 y 통과
endmodule
```

합성 회로도:

```
x ──→┤bufif0├──→┐
     └───────┘    ├──→ f (tri-state bus)
y ──→┤bufif1├──→┘
     └───────┘

s ──→──┤ctrl│
```

Array of Instances

```
wire [3:0] out, in1, in2;
nand n_gate[3:0](out, in1, in2);  // 4개 nand를 한 번에
// 동치: nand n_gate0(out[0], in1[0], in2[0]); ... n_gate3(out[3],
in1[3], in2[3]);
```

합성 도구 의존적이므로 사용 전 확인 필요.

3. Gate Instantiation 예제들

기본 게이트 조합:

```
module basic_gates(input x, y, z, output f);
  wire a, b, c;
  nor g1(b, x, y);
  not g2(a, x);
  and g3(c, a, z);
  nor g4(f, b, c);
endmodule
```

합성 회로도:

```
x ──→──┤ NOR ├──→ b ──→──┤      │
y ──→──┤ g1  │            │ NOR ├──→ f
       └─────┘            │ g4  │
x ──→──┤ NOT ├──→ a ──┐   │     │
       │ g2  │        │   └─────┘
       └─────┘        │
z ──────────────────┤ AND ├──→ c ──→ g4
                    │ g3  │
              a ──→──┤     │
                    └─────┘
```

1-bit Full Adder (구조적):

```
module full_adder_structural(input x, y, c_in,
                             output s, c_out);
  wire s1, c1, c2, c3;
  xor xor_s1(s1, x, y);
  xor xor_s2(s, s1, c_in);
```

```
  and and_c1(c1, x, y);
  and and_c2(c2, x, c_in);
  and and_c3(c3, y, c_in);
  or  or_cout(c_out, c1, c2, c3);
endmodule
```

합성 회로도:

```
x ──→──┤XOR├──→ s1 ──→──┤XOR├──→ s
y ──→──┤s1 │            │s2 │
       └───┘       c_in──→│   │
                          └───┘

x ──→──┤AND├──→ c1 ──┐
y ──→──┤   │         │
       └───┘         │
x ──→──┤AND├──→ c2 ──┼──→┤ OR  ├──→ c_out
c_in──→┤   │         │   │     │
       └───┘         │   └─────┘
y ──→──┤AND├──→ c3 ──┘
c_in──→┤   │
       └───┘
```

4:1 Multiplexer (구조적):

```
module mux4_to_1_structural(input i0, i1, i2, i3, s1, s0,
                            output out);
  wire s1n, s0n, y0, y1, y2, y3;
  not(s1n, s1);
  not(s0n, s0);
  and(y0, i0, s1n, s0n);   // i0·s¯1·s¯0
  and(y1, i1, s1n, s0);    // i1·s¯1·s0
  and(y2, i2, s1, s0n);    // i2·s1·s¯0
  and(y3, i3, s1, s0);     // i3·s1·s0
  or(out, y0, y1, y2, y3);
endmodule
```

합성 회로도:

```
i0──→┤AND├──→y0─┐
s1n─→┤   │      │
s0n─→┤   │      │
     └───┘      │
i1──→┤AND├──→y1─┤
s1n─→┤   │      ├──→┤ OR  ├──→ out
s0 ─→┤   │      │   │     │
     └───┘      │   └─────┘
i2──→┤AND├──→y2─┤
s1 ─→┤   │      │
s0n─→┤   │      │
     └───┘      │
i3──→┤AND├──→y3─┘
s1 ─→┤   │
s0 ─→┤   │
     └───┘
```

9-bit Parity Generator (구조적 vs 데이터플로우):

```
// Structural: 트리 구조의 XOR 게이트
module parity_gen_9b_structural(input [8:0] x, output ep, op);
  wire c, d, e, f, g, h, j;
  xor xor_11(c, x[0], x[1]);  xor xor_12(d, x[2], x[3]);
  xor xor_13(e, x[4], x[5]);  xor xor_14(f, x[6], x[7]);
```

```
  xor xor_21(g, c, d);          xor xor_22(h, e, f);
  xor xor_31(i, g, h);
  xor xor_ep(ep, i, x[8]);
  xnor xnor_op(op, i, x[8]);
endmodule

// Dataflow: 한 줄!
module parity_gen_9b_reduction(input [8:0] x, output ep, op);
  assign ep = ^x;
  assign op = ~ep;
endmodule
```

합성 회로도 (트리 구조):

```
x[0]──→┤XOR├─c─┐
x[1]──→┤    │   ├─→┤XOR├─g─┐
x[2]──→┤XOR├─d─┘  │   │   ├─→┤XOR├─i─→┤XOR ├──→ ep
x[3]──→┤    │     └───┘   │  │   │    │    │
x[4]──→┤XOR├─e─┐           │  └───┘  x[8]──→┤
x[5]──→┤    │   ├─→┤XOR├─h─┘
x[6]──→┤XOR├─f─┘  │   │         i──→┤XNOR├──→ op
x[7]──→┤    │     └───┘        x[8]→┤    │
       └────┘                        └────┘
```

4. Net Types

wire/tri: 기능 동일. 여러 드라이버가 같은 값이면 그 값, 다른 값(0과 1)이면 x, 한쪽이 z이면 다른 쪽 값이 이김.

wire/tri	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

wand/triand: 하나라도 0이면 결과 0 (Wired AND). wor/trior: 하나라도 1이면 결과 1 (Wired OR). tri0: 미구동 시 0 (pull-down). tri1: 미구동 시 1 (pull-up).

Wired AND 예제 (Open-Drain):

```
module open_drain(input w, x, y, z, output f);
  wand f;
  nand n1 (f, w, x);
  nand n2 (f, y, z);
endmodule
// f = (wx)¯ · (yz)¯
```

합성 회로도:

```
w──→┤NAND├──→┐
x──→┤ n1 │    ├──→ f (wand: AND 해석)
    └────┘    │
y──→┤NAND├──→┘
z──→┤ n2 │
    └────┘
```

5. Delays

Gate delay: 입력→출력 전파 시간 (inertial delay model, 짧은 펄스 무시).
Wire delay: 기생 R,C에 의한 지연 (transport delay model, 모든 변화 전파).

Gate Delay 지정:

```
and #(5) a1(b, x, y);                          // 단일 지연
not #(10:12:15) n1(a, x);                       // min:typ:max
and #(10:12:15, 12:15:20) a2(c, a, z);         // rise, fall
or  #(10:12:15, 12:15:20, 12:13:16) o2(f,b,c);  // rise, fall, turn-
off
```

Rise delay: 0/x/z → 1. Fall delay: 1/x/z → 0. Turn-off delay: 0/1/x → z.

생략 규칙: 값 없음→모두 0. 1개→모든 전이 동일. 2개(rise,fall)→turn-off = min(rise,fall).

지연 계산 테이블 (IEEE Std 1364-2005):

From→To	1개 지정	2개 지정(r,f)	3개 지정(r,f,off)
0 → 1	d1	d1	d1
0 → x	d1	min(d1,d2)	min(d1,d2,d3)
0 → z	d1	min(d1,d2)	d3
1 → 0	d1	d2	d2
1 → x	d1	min(d1,d2)	min(d1,d2,d3)
1 → z	d1	min(d1,d2)	d3
x → 0	d1	d2	d2
x → 1	d1	d1	d1
x → z	d1	min(d1,d2)	d3
z → 0	d1	d2	d2
z → 1	d1	d1	d1
z → x	d1	min(d1,d2)	min(d1,d2,d3)

6. Hazards

입력→출력 경로의 전파 지연이 서로 다를 때 출력이 안정 전에 일시적으로 흔들리는 현상.

Static hazard: 출력이 일정해야 하는데 잠깐 반대 값으로 갔다가 복귀. - Static-1 hazard: 1이어야 하는데 잠깐 0 (0 glitch) - Static-0 hazard: 0이어야 하는데 잠깐 1 (1 glitch)

Dynamic hazard: 출력이 한 번 변해야 하는데 3번 이상 변함. 경로 3개 이상 필요.

Static Hazard 예제:

```
module hazard_static(input x, y, z, output f);
  wire a, b, c;
  and #5 a1(a, x, y);
  not #5 n1(c, x);
  and #5 a2(b, c, z);
  or  #5 o2(f, b, a);
endmodule
```

f = xy + $\bar{x}$z. y=1,z=1에서 x가 1→0:

```
x:      ──────┐
              └──────
a(xy):  ──────┐        (5단위 후 0으로)
              └──
c(x̄):   ──────────┐    (5단위 후 1로)
```

```
                    └──
  b(x̄z): ──────────────┐ (c 이후 5단위 후 1로)
                       └──
  f:     ────────────┐  ┌── (a가 먼저 0 → b 아직 0 → f 잠깐 0!)
                     └──┘   ← Static-1 Hazard!
```

K-Map으로 발견 및 제거:

```
    x
yz   0  1
 00 |0 |0
 01 |1 |0    인접한 1(yz=11, x=0과 x=1)이
 11 |1 |1  ← 같은 루프에 없음 → hazard!
 10 |0 |1

해결: f = xy + x̄z + yz (중복 항 yz 추가)
```

Dynamic Hazard 예제:

```
module hazard_dynamic(input w, x, y, z, output f);
  wire a, b, c, d, e;
  nand #5 nand1(b, x, w);
  not  #5 n1(a, w);
  nand #5 nand2(c, a, y);
  nand #5 nand3(d, b, c);
  nand #5 nand4(e, w, z);
  nand #5 nand5(f, d, e);
endmodule
```

w에서 f까지 3개 경로(게이트 2개/3개/4개)의 지연 차이 → f가 여러 번 변하는 dynamic hazard 발생.

Ch6: Tasks, Functions, UDPs

이 챕터에서는 Verilog 코드의 재사용성과 가독성을 높이기 위한 세 가지 메커니즘을 다룬다: Functions, Tasks, User-Defined Primitives(UDPs). 핵심은 Function과 Task의 차이를 정확히 이해하는 것이다. Function은 조합 논리 전용이고, Task는 타이밍 제어를 포함할 수 있어 더 범용적이다. UDP는 자신만의 게이트 프리미티브를 정의하는 방법인데, 합성 도구에서 보통 지원하지 않는다.

1. Functions vs Tasks — 핵심 비교

```
┌──────────────────────┬──────────────────────────┬──────────────
│ 항목                 │ Function                 │ Task
│
├──────────────────────┼──────────────────────────┼──────────────
│ 인자                 │ input만, output/inout 불가 │ input, output,
inout 모두    │
│ 반환값               │ 함수 이름으로 단일 값      │ output/inout으로
여러 값     │
│ 타이밍 제어          │ 불가 (#delay, posedge 등)  │ 가능
│
│ 실행 시간            │ 0 시뮬레이션 시간          │ non-zero 가능
│
│ 호출 가능 대상       │ function만                 │ function +
task 모두         │
```

```
│ 표현식 사용          │ 가능 (assign y = func(x))    │ 불가
│
│ 용도                 │ 조합 논리 전용!              │ 범용
│
└────────────────────┴──────────────────────────┴──────────────
```

직관: Function = 수학 함수 (입력→즉시 결과), Task = 서브루틴 (시간 소요 가능, 여러 결과).

2. Functions — 상세

선언: function [automatic][signed][return_type] name (input [port_list]); ... endfunction

규칙: (1) input만, output/inout 불가 (2) 함수 이름에 반환값 할당 (3) 타이밍 제어 불가 → 조합 논리만! (4) 로컬 변수 (5) 전역 신호 구동 가능 (6) function만 호출 가능 (task 불가) (7) 표현식에서 사용 가능

Zero Counter 예제:

```
module zero_count_function(input [7:0] data,
                           output reg [3:0] out);
  always @(data)
    out = count_0s_in_byte(data);

  function [3:0] count_0s_in_byte(input [7:0] data);
    integer i;
    begin
      count_0s_in_byte = 0;
      for (i = 0; i <= 7; i = i + 1)
        if (data[i] == 0)
          count_0s_in_byte = count_0s_in_byte + 1;
    end
  endfunction
endmodule
```

count_0s_in_byte는 함수 이름이자 반환 변수이다. 합성 결과는 Ch4의 for 루프 버전과 동일 (8개 비교기+가산기 체인).

Static vs Automatic

C에서는 기본이 automatic, 필요하면 static. Verilog는 반대: 기본이 static, 필요하면 automatic. automatic은 재귀 호출과 동시 실행에 필요.

Factorial (Automatic Function):

```
module factorial(input [7:0] n, output [15:0] result);
  assign result = fact(n);

  function automatic [15:0] fact;
    input [7:0] N;
    if (N == 1) fact = 1;
    else fact = N * fact(N - 1);   // 재귀 호출!
  endfunction
endmodule
```

automatic 없으면 N이 정적 메모리 공유 → 재귀 오동작.

Constant Function (elaboration time 평가):

```
module RAM #(parameter RAM_Depth = 1024) (
  input [count_log_b2(RAM_Depth)-1:0] addr_bus,
  output reg [7:0] data_bus);

  function integer count_log_b2(input integer depth);
    begin
      count_log_b2 = 0;
      while (depth) begin
        count_log_b2 = count_log_b2 + 1;
        depth = depth >> 1;
      end
    end
  endfunction
endmodule
// count_log_b2(1024) = 10 → addr_bus는 [9:0]
```

3. Tasks — 상세

선언: task [automatic] name (input/inout/output [port_list]); … endtask

규칙: (1) input, output, inout 모두 가능 (2) 타이밍 제어 가능 (#delay, posedge 등) (3) 로컬 변수 (4) 전역 신호 구동 가능 (5) function + task 모두 호출 가능 (6) 표현식에서 사용 불가

Zero Counter (Task 버전):

```
module zero_count_task(input [7:0] data,
                       output reg [3:0] out);
  always @(data)
    count_0s_in_byte(data, out);

  task count_0s_in_byte(input [7:0] data, output reg [3:0] count);
    integer i;
    begin
      count = 0;
      for (i = 0; i <= 7; i = i + 1)
        if (data[i] == 0) count = count + 1;
    end
  endtask
endmodule
```

Function과 차이: function은 out = func(data), task는 task_name(data, out).

전역 변수 구동 + 타이밍 제어:

```
task do_write;
  input [7:0] i_addr, i_data;
  begin
    addr_valid = 1'b1;          // 전역 변수 직접 구동
    mux_addr_data = i_addr;
    #10;                         // 타이밍 제어 (function 불가!)
    addr_valid = 1'b0;
    data_valid = 1'b1;
    mux_addr_data = i_data;
  end
endtask
```

Static vs Automatic Task:

Static task (기본):

```
5  At beginning, count = x     ← 첫 호출
5  After reset, count = 0
12 At end, count = 2
32 At beginning, count = 2     ← 두 번째: 이전 값 2 남아있음!
39 At end, count = 4
```

Automatic task:

```
5  At beginning, count = x     ← 첫 호출
5  After reset, count = 0
12 At end, count = 2
32 At beginning, count = x     ← 두 번째: 새 변수(x)!
39 At end, count = x
```

4. User-Defined Primitives (UDPs)

사용자가 자신만의 게이트 프리미티브를 정의. 조합 UDP와 순차 UDP 두 종류.

기본 규칙: scalar 포트만, 하나의 출력만(첫 포트), 순차 UDP는 output reg, z 미지원(x로 처리), 합성 보통 불가.

Combinational UDP — AND:

```
primitive udp_and(out, a, b);
  output out;
  input a, b;
  table
    // a b : out;
    0 0 : 0;
    0 1 : 0;
    1 0 : 0;
    1 1 : 1;
  endtable
endprimitive
```

Don't care: ?는 0,1,x. 예: `0 ? : 0;` (a=0이면 b 무관하게 out=0)

Sequential UDP — D Latch:

```
primitive d_latch(output reg q, input d, gate, clear);
  initial q = 0;
  table
    // d  gate  clear : q : q+;
    ?    ?     1     : ? : 0 ;    // clear 우선
    1    1     0     : ? : 1 ;    // gate=1 → latch
    0    1     0     : ? : 0 ;
    ?    0     0     : ? : - ;    // gate=0 → no change
  endtable
endprimitive
```

`-` = 상태 변화 없음 (순차 UDP 출력 전용).

Sequential UDP — T Flip-Flop:

```
primitive T_FF(output reg q, input clk, clear);
```

```
  table
    // clk    clear : q : q+;
    ?       1     : ? : 0 ;    // 비동기 clear
    ?       (10)  : ? : - ;    // clear fall 무시
    (01)    0     : 1 : 0 ;    // clk rise → toggle
    (01)    0     : 0 : 1 ;
    (1?)    0     : ? : - ;    // clk fall 무시
  endtable
endprimitive
```

(01)=rising edge, (10)=falling edge, (1?)=1에서 0/1/x.

Shorthand Symbols:

```
?  = 0, 1, x       b = 0, 1        - = no change (순차 출력 전용)
r  = (01)           f = (10)        p = (01),(0x),(x1)
n  = (10),(1x),(x0) * = (??)
```

Ripple Counter:

```
module ripple_counter(input clock, clear, output [3:0] q);
  T_FF tff0(q[0], clock, clear);
  T_FF tff1(q[1], q[0], clear);
  T_FF tff2(q[2], q[1], clear);
  T_FF tff3(q[3], q[2], clear);
endmodule
```

합성 회로도:

```
clock──→┤T_FF├──→q[0]──→┤T_FF├──→q[1]──→┤T_FF├──→q[2]──→┤T_FF├──→q[3]

          │tff0│          │tff1│          │tff2│          │tff3│
  clear──→│    │  clear──→│    │  clear──→│    │  clear──→│    │
          └────┘          └────┘          └────┘          └────┘
```

UDP 설계 가이드라인: 기능만 모델링(타이밍 아님), scalar 출력 1개, 합성 도구 보통 미지원, 테이블 완전 정의 권장, shorthand 활용.

Ch7: Hierarchical Modeling

이 챕터의 핵심은 파라미터(parameter)와 generate 블록이다. 동일한 기능의 모듈을 비트 폭이나 지연 값만 바꿔서 재사용하려면, 하드코딩 대신 파라미터로 만들어 인스턴스화 시점에 오버라이드하면 된다. generate 블록은 elaboration time(컴파일 시점)에 하드웨어를 반복 생성하거나 조건부로 선택하는 메커니즘으로, 파라미터화된 설계의 핵심 도구이다.

1. Parameters — 세 가지 종류

parameter: 가장 일반적. 지연/폭 지정. defparam이나 #()으로 오버라이드 가능.

```
parameter SIZE = 7;
parameter WIDTH_BUSA = 24, WIDTH_BUSB = 8;
```

localparam: 모듈 내부 전용. 외부에서 오버라이드 불가. 파생 상수에 사용.

```
parameter word_size = 16;
localparam memory_size = word_size * 512;  // 자동 업데이트
```

specparam: specify 블록 내 타이밍 파라미터. 이 과목에서는 깊게 안 다룸.

Parameter Port 선언 (ANSI 스타일):

```
module module_name
  #(parameter SIZE = 7,
    parameter WIDTH_BUSA = 24)
  (port list);
endmodule
```

2. Parameter Override — 두 가지 방법

defparam (비추천):

```
defparam cnt_4b.N = 4, cnt_8b.N = 8;
counter_nbits cnt_4b(clock, clear, qout4b);
counter_nbits cnt_8b(clock, clear, qout8b);
```

#() 인스턴스화 (추천!):

```
counter_nbits #(4) cnt_4b(clock, clear, qout4b);   // positional
counter_nbits #(8) cnt_8b(clock, clear, qout8b);

// named association (더 안전!)
hazard_static #(.delay2(4), .delay1(6)) example2(x, y, z, f);
```

N-bit Counter 예제:

```
module counter_nbits
  #(parameter N = 4)
  (input clock, clear,
   output reg [N-1:0] qout);
  always @(posedge clock or negedge clear)
    if (clear) qout <= {N{1'b0}};
    else       qout <= (qout + 1);
endmodule
```

합성 회로도:

```
[cnt_4b: N=4]                    [cnt_8b: N=8]
qout[3:0]──→┤+├──→┤DFF[3:0]├    qout[7:0]──→┤+├──→┤DFF[7:0]├
       1──→─┤ │    │ R      │            1──→─┤ │    │ R      │
            └─┘    └────────┘                 └─┘    └────────┘
→ 같은 모듈에서 파라미터만 바꿔 4비트/8비트 카운터!
```

3. Parameterized Module 예제들

Parameterized Decoder (n-to-m):

```
module decoder_n2m_high
  #(parameter n = 3, parameter m = 8)
  (input [n-1:0] x, input enable,
   output reg [m-1:0] y);
  always @(x, enable)
    if (!enable) y = {m{1'b0}};
    else y = {{m-1{1'b0}}, 1'b1} << x;  // 1을 x만큼 좌시프트!
endmodule
```

핵심 트릭: {{m-1{1'b0}}, 1'b1}은 m비트 0...01을 만들고, <<x로 x번째 위치에 1을 놓는다.

Parameterized Mux (M-to-1):

```
module mux_m_to_1
  #(parameter M = 4, parameter K = 2)
  (input [K-1:0] select, input [M-1:0] in,
   output reg y);
  integer i;
  always @(*)
    for (i = 0; i < M; i = i + 1)
      if (select == i) y = in[i];
endmodule
```

4. Hierarchical Names

```
4bit_adder.fa_1.ha_1.xor1.s    // 최하위 넷까지 참조
```

5. generate Block

elaboration time에 하드웨어 생성. genvar는 시뮬레이션 중 존재하지 않음. block name 필수.

generate에 넣을 수 있는 것: net/var 선언, module 인스턴스화, assign, always, function, task. 넣을 수 없는 것: parameter/localparam 선언, input/output/inout, specify.

세 형태: generate loop(for), generate conditional(if...else), generate case.

Gray-to-Binary 변환:

```
module gray2bin1 #(parameter SIZE = 8)
  (input [SIZE-1:0] gray, output [SIZE-1:0] bin);
  genvar i;
  generate
    for (i = 0; i < SIZE; i = i + 1) begin: bit
      assign bin[i] = ^gray[SIZE-1:i];
    end
  endgenerate
endmodule
```

n-bit Ripple Carry Adder (generate + full_adder):

```
module adder_nbit #(parameter N = 4)
  (input [N-1:0] x, y, input c_in,
   output [N-1:0] sum, output c_out);
  genvar i;
  wire [N-2:0] c;
  generate
    for (i = 0; i < N; i = i + 1) begin: adder
      if (i == 0)
        full_adder fa(x[i], y[i], c_in, sum[i], c[i]);
      else if (i == N-1)
        full_adder fa(x[i], y[i], c[i-1], sum[i], c_out);
      else
        full_adder fa(x[i], y[i], c[i-1], sum[i], c[i]);
    end
```

```
    endgenerate
  endmodule
```

합성 회로도 (N=4):

```
  x[0]──→┤FA├──→sum[0]   x[1]──→┤FA├──→sum[1]   x[2]──→┤FA├──→sum[2]
x[3]──→┤FA├──→sum[3]
  y[0]──→┤  │            y[1]──→┤  │            y[2]──→┤  │
y[3]──→┤  │

c_in──→┤ci├──→c[0]──→──┤ci├──→c[1]──→──┤ci├──→c[2]──→──┤ci├──→c_out
        └──┘             └──┘             └──┘             └──┘
```

2's Complement Adder/Subtractor (generate):

```
  module twos_adder_nbit #(parameter N = 4)
    (input [N-1:0] x, y, input mode,
     output [N-1:0] sum, output c_out);
    genvar i;
    wire [N-2:0] c;
    wire [N-1:0] t;
    generate
      for (i = 0; i < N; i = i + 1) begin: ones_complement_generator
        xor xor_oc(t[i], y[i], mode);
      end
    endgenerate
    generate
      for (i = 0; i < N; i = i + 1) begin: adder
        if (i == 0)      full_adder fa(x[i], t[i], mode, sum[i],
c[i]);
        else if (i == N-1) full_adder fa(x[i], t[i], c[i-1], sum[i],
c_out);
        else             full_adder fa(x[i], t[i], c[i-1], sum[i],
c[i]);
      end
    endgenerate
  endmodule
```

합성 회로도:

```
  y[i]──→┤XOR├──→t[i]──→┤FA├──→sum[i]
  mode──→┤   │          │  │
         └───┘     x[i]──→┤  ├──→c[i]
                   c[i-1]→┤ci│
                          └──┘

  mode=0: t=y, c_in=0 → x+y (덧셈)
  mode=1: t=~y, c_in=1 → x-y (뺄셈)
```

generate case:

```
  generate
    for (i = 0; i < N; i = i + 1) begin: adder
      case(i)
        0:   assign {c[i], sum[i]} = x[i] + y[i] + c_in;
        N-1: assign {c_out, sum[i]} = x[i] + y[i] + c[i-1];
        default: assign {c[i], sum[i]} = x[i] + y[i] + c[i-1];
      endcase
    end
  endgenerate
```

Ch8: Sequential Logic Modules

이 챕터는 실제 디지털 시스템에서 가장 많이 사용되는 순차 논리 빌딩 블록들을 총정리한다. 크게 다섯 부분이다: (1) 레지스터와 메모리, (2) 시프트 레지스터, (3) 카운터, (4) 시퀀스 생성기(LFSR, Ring, Johnson), (5) 파이프라이닝과 Signed Arithmetic.

1. D Flip-Flop — Async vs Sync Reset

Asynchronous Reset: 리셋이 sensitivity list에 포함. 클럭 무관하게 즉시 동작.

```
module DFF_async_reset(input clk, reset_n, d, output reg q);
  always @(posedge clk or negedge reset_n)
    if (!reset_n) q <= 0;    // 즉시 리셋
    else          q <= d;
endmodule
```

합성 회로도:

```
d──→┤D  Q├──→q      reset_n이 0이 되는 순간
    │DFF │          클럭과 무관하게 q=0
clk→┤>CLR├
reset_n→┤ │
    └────┘
```

Synchronous Reset: 리셋이 sensitivity list에 없음. 클럭 에지에서만 리셋 확인.

```
module DFF_sync_reset(input clk, reset_n, d, output reg q);
  always @(posedge clk)       // reset_n 없음!
    if (!reset_n) q <= 0;     // clk edge에서만 확인
    else          q <= d;
endmodule
```

합성 회로도:

```
d──→┤1│MUX├──→┤D  Q├──→q     MUX로 d와 0 중 선택
0──→┤0│   │   │DFF │
    └─┘   │   └────┘
reset_n→sel   clk→┤>│
```

2. Data Registers

기본 N-bit:

```
module register #(parameter N = 4)
  (input clk, input [N-1:0] din, output reg [N-1:0] qout);
  always @(posedge clk) qout <= din;
endmodule
```

Sync Load + Async Reset:

```
module register_load_reset #(parameter N = 4)
  (input clk, load, reset_n, input [N-1:0] din,
   output reg [N-1:0] qout);
  always @(posedge clk or negedge reset_n)
    if (!reset_n)    qout <= {N{1'b0}};
    else if (load)   qout <= din;    // load=0이면 값 유지
endmodule
```

합성 회로도:

```
din──→┤1│MUX├──→┤D  Q├──→qout
qout─→┤0│   │   │DFF │
      └─┘   │   │CLR │
load→sel     clk→┤>│  reset_n→CLR
```

3. Register File / SRAM (1W2R)

```
module register_file
  #(parameter M=4, N=16, W=8)
  (input clk, we, input [W-1:0] din,
   output [W-1:0] dout_a, dout_b,
   input [M-1:0] raddr_a, raddr_b, waddr);
  reg [M-1:0] raddr_a_buf, raddr_b_buf, waddr_buf;
  reg [W-1:0] din_buf; reg we_buf;
  reg [W-1:0] mem [N-1:0];
  assign dout_a = mem[raddr_a_buf];
  assign dout_b = mem[raddr_b_buf];
  always @(posedge clk) begin
    raddr_a_buf<=raddr_a; raddr_b_buf<=raddr_b;
    waddr_buf<=waddr; we_buf<=we; din_buf<=din;
    if (we_buf) mem[waddr_buf] <= din_buf;
  end
endmodule
```

입력을 DFF로 버퍼링한 후 메모리 접근. FPGA에서는 LUT RAM/Block RAM으로 합성.

4. Shift Registers

기본 N-bit:

```
module shift_register #(parameter N = 4)
  (input clk, reset_n, din, output reg [N-1:0] qout);
  always @(posedge clk or negedge reset_n)
    if (!reset_n) qout <= {N{1'b0}};
    else          qout <= {din, qout[N-1:1]};  // 우시프트
endmodule
```

합성 회로도:

```
din──→┤DFF├──→qout[3]──→┤DFF├──→qout[2]──→┤DFF├──→qout[1]──→┤DFF├──→(

        └───┘               └───┘               └───┘
└───┘
                                    clk (공유)
```

Universal Shift Register (s1,s0으로 4가지 모드):

```
s1 s0 │ Function
──────┼──────────
0  0  │ No change
0  1  │ Right shift (rsi→MSB→...→LSB)
1  0  │ Left shift  (lsi→LSB→...→MSB)
1  1  │ Parallel load (din→qout)
```

```
module universal_shift_register #(parameter N = 4)
  (input s1, s0, lsi, rsi, clk, reset_n,
```

```
  input [N-1:0] din, output reg [N-1:0] qout);
 always @(posedge clk or negedge reset_n)
   if (!reset_n) qout <= {N{1'b0}};
   else case ({s1, s0})
     2'b00: ;                              // no change
     2'b01: qout <= {rsi, qout[N-1:1]};   // right shift
     2'b10: qout <= {qout[N-2:0], lsi};   // left shift
     2'b11: qout <= din;                   // parallel load
   endcase
endmodule
```

합성 회로도 (각 비트):

```
qout[i]  ──→┤0│         (no change)
qout[i+1]─→┤1│ 4:1      (right shift)
qout[i-1]─→┤2│ MUX──→┤DFF├──→qout[i]
din[i]   ─→┤3│        └───┘
            └─┘  clk→┤>│
{s1,s0}──→sel
```

5. Counters

Ripple Counter (비동기, generate):

```
module ripple_counter_generate #(parameter N = 4)
  (input clk, reset_n, output reg [N-1:0] qout);
  genvar i;
  generate for (i=0; i<N; i=i+1) begin: rc
    if (i==0)
      always @(negedge clk or negedge reset_n)
        if (!reset_n) qout[0]<=0; else qout[0]<=~qout[0];
    else
      always @(negedge qout[i-1] or negedge reset_n)
        if (!reset_n) qout[i]<=0; else qout[i]<=~qout[i];
  end endgenerate
endmodule
```

합성 회로도:

```
clk─→neg┤TFF├─→q[0]─→neg┤TFF├─→q[1]─→neg┤TFF├─→q[2]─→neg┤TFF├─→q[3]
```

Binary Counter (동기):

```
module binary_counter #(parameter N = 4)
  (input clk, enable, reset,
   output reg [N-1:0] qout, output cout);
  always @(posedge clk)
    if (reset)       qout <= {N{1'b0}};
    else if (enable) qout <= qout + 1;
  assign #1 cout = &qout;  // carry: all 1s
endmodule
```

Up/Down Counter + Cascade:

```
module up_dn_bin_counter #(parameter N = 4)
  (input clk, reset, eup, edn,
   output reg [N-1:0] qout, output cout, bout);
  always @(posedge clk)
    if (reset)    qout <= {N{1'b0}};
    else if (eup) qout <= qout + 1;
```

```
        else if (edn) qout <= qout - 1;
      assign #1 cout = (&qout) & eup;
      assign #1 bout = (~|qout) & edn;
    endmodule

    // 8-bit cascade
    up_dn_bin_counter #(4)
cnt1(clk,reset,eup,edn,qout[3:0],cout1,bout1);
    up_dn_bin_counter #(4)
cnt2(clk,reset,cout1,bout1,qout[7:4],cout,bout);
```

Mod-R Counter (R=10이면 BCD):

```
    module modulo_r_counter #(parameter N=4, R=10)
      (input clk, enable, reset,
       output reg [N-1:0] qout, output reg cout);
      wire cx;
      assign cx = (qout == R-1);
      always @(negedge clk) cout <= cx;
      always @(posedge clk)
        if (reset)       qout <= {N{1'b0}};
        else if (enable)
          if (cout) qout <= 0; else qout <= qout + 1;
    endmodule
```

6. Sequence Generators

LFSR (Standard Form, 의사 난수):

```
    module pr_sequence_generate
      #(parameter N=4, parameter [N:0] tap=5'b10011)
      (input clk, start, output reg [N-1:0] qout);
      wire d;
      assign d = ^(tap[N-1:0] & qout[N-1:0]);
      always @(posedge clk or posedge start)
        if (start) qout <= {1'b1, {N-1{1'b0}}};
        else       qout <= {d, qout[N-1:1]};
    endmodule
```

합성 회로도:

```
qout[0]──→┤XOR├──→d──→┤DFF├──→qout[3]──→┤DFF├──→qout[2]──→┤DFF├──→qou

  qout[1]──→┤    |        └──┘                └──┘                └──┘
└──┘
             └──┘
  tap=10011 → x⁴+x+1, 시퀀스 길이 2⁴-1=15
```

최대 주기 조건: 다항식이 primitive polynomial이어야 함.

Ring Counter (N비트 N상태, one-hot):

```
    module ring_counter #(parameter N = 4)
      (input clk, start, output reg [0:N-1] qout);
      always @(posedge clk or posedge start)
        if (start) qout <= {1'b1, {N-1{1'b0}}};
        else       qout <= {qout[N-1], qout[0:N-2]};
    endmodule
    // 시퀀스: 1000→0100→0010→0001→1000
```

Johnson Counter (N비트 2N상태, 반전 피드백):

```
module johnson_counter #(parameter N = 4)
  (input clk, start, output reg [0:N-1] qout);
  always @(posedge clk or posedge start)
    if (start) qout <= {N{1'b0}};
    else       qout <= {~qout[N-1], qout[0:N-2]};
endmodule
// 시퀀스: 0000→1000→1100→1110→1111→0111→0011→0001→0000
```

7. Timing Generators

Ring counter: N-bit → N개 비중첩 타이밍. Binary counter + decoder: M-bit counter → 2^M개 타이밍 (FF 적음).

```
module binary_counter_timing_generator
  #(parameter N=8, M=3)
  (input clk, reset, enable, output reg [N-1:0] qout);
  reg [M-1:0] bcnt_out;
  always @(posedge clk or posedge reset)
    if (reset) bcnt_out <= {M{1'b0}};
    else if (enable) bcnt_out <= bcnt_out + 1;
  always @(bcnt_out)
    qout = {{N-1{1'b0}}, 1'b1} << bcnt_out;
endmodule
```

8. Pipelining

Critical path가 클럭 주기보다 길 때, 로직을 나누어 여러 사이클에 처리. 핵심: 모든 중간 신호를 DFF로 지연!

```
module adder_pipelined(input clk, input [15:0] a, b,
                       output reg [15:0] result);
  reg [7:0] partial_sum, a_msb, b_msb;
  reg partial_cout;
  always @(posedge clk) begin
    a_msb <= a[15:8]; b_msb <= b[15:8];      // Stage 1: MSB 지연
    {partial_cout, partial_sum} <= a[7:0] + b[7:0];  // Stage 1: LSB
덧셈
    result <= {a_msb + b_msb + partial_cout, partial_sum}; // Stage
2
  end
endmodule
```

합성 회로도:

```
  Stage 1                          Stage 2

a[7:0]──→┤ADD├──→partial_sum──→┤DFF├──→──────────────────────→result[

  b[7:0]──→┤   ├──→partial_cout─→┤DFF├──→┤ADD├──→result[15:8]
           └───┘                          │   │
  a[15:8]──→┤DFF├──→a_msb──────────────→┤   │
  b[15:8]──→┤DFF├──→b_msb──────────────→┤   │
                                          └───┘
```

9. Signed Arithmetic (Verilog 2001)

핵심 규칙: 1. 모든 연산은 기본 unsigned 2. 모든 피연산자가 signed여야 signed 연산. 하나라도 unsigned이면 전체 unsigned! 3. signed 연산에서 피연산자는 sign-extended됨 4. LHS 타입은 연산에 영향 없음

signed/unsigned 캐스팅:

```
A = $unsigned(B);   // zero-fill
A = $signed(B);     // sign-extend
```

Signed 덧셈:

```
// signed + signed → 자동 sign extension
assign sum = A + B;

// signed + unsigned carry → $signed 캐스팅!
assign sum = A + B + $signed({1'b0, carry_in});
```

Signed 곱셈:

```
// signed × unsigned → $signed 필요!
assign prod = a * $signed({1'b0, b});
```

항상 unsigned인 연산: bit-select, part-select, concatenation, comparison 결과.

Signed number 예제:

```
a = 14'h1234;    // → 00001234 (zero-extend)
a = -14'h1234;   // → ffffedcc (2의 보수)
a = 14'sh2dcc;   // → ffffffdcc (sign-extend! MSB=1 → 음수)
a = 14'h2dcc;    // → 00002dcc (zero-extend)
```

Ch9: Design Options of Digital Systems

이 챕터는 Verilog 코딩이 아니라, 디지털 시스템을 실제로 구현할 때 어떤 하드웨어 플랫폼을 선택할 수 있는지를 다룬다. 크게 ASIC(맞춤형 칩)과 Field-Programmable Devices(현장 프로그래밍 가능 장치)로 나뉜다.

1. 구현 옵션 분류 및 비용 구조

```
디지털 시스템 설계 옵션
├── ASIC
│   ├── Full-Custom
│   ├── Gate Array (GA)
│   └── Standard-Cell Based
└── Field-Programmable
    ├── PLD (ROM / PLA / PAL)
    ├── CPLD
    └── FPGA
```

Total Cost = NRE Cost + Unit Cost × Volume. - NRE: 일회성 개발 비용. ASIC 높음($100K~$1M+), FPGA 거의 0. - Unit Cost: 단위 칩 비용. ASIC 낮음, FPGA 높음. - 교차점 k: k 이하에서는 FPGA 유리, k 이상에서는 ASIC 유리. 최근 k가 $_{10K\rightarrow}$100K로 이동.

왼쪽(PLD)→오른쪽(Full-custom)으로 갈수록: Design flexibility/Speed/NRE 증가, Time to market/Ease of use 감소.

2. ASIC

최고의 에너지 효율/성능. 제조 비용 높고, 턴어라운드 3~6개월, 디버그 불가(재제조 필요).

Full-Custom: 모든 트랜지스터/와이어 레이아웃 직접 설계. 최고 성능, 최고 NRE.

Gate Array: p/n 트랜지스터 미리 배치. 메탈 레이어만 커스터마이즈. NRE 낮음.

Standard-Cell Based: 미리 설계된 표준 셀 라이브러리 사용. 고정 높이, 가변 너비 셀을 행 단위 배치.

Standard Cell Library 포함 내용: - 조합 셀: Inverter/Buffer(1X~16X), NAND/AND/NOR/OR/XOR(2~8 input, high/normal/low power), MUX/Decoder - 순차 셀: D/JK Latch, D/JK FF (sync/async reset), Counter - 서브시스템: Adder(CLA/Brent-Kung/Kogge-Stone), Multiplier, Barrel Shifter - I/O 패드 - 각 셀에: Behavioral model, SPICE model, Delay/Power/Area, Layout, Abstract model

3. PLD (Programmable Logic Device)

2-level AND-OR 배열로 SOP 논리 구현. Fuse(연결→끊기) 또는 Anti-fuse(미연결→연결).

ROM: AND 고정(모든 minterm) + OR 프로그래밍. $2^n \times m$ ROM은 m개의 n-변수 함수 구현. 진리표 전체 저장과 동일.

PLA: AND + OR 모두 프로그래밍. ROM보다 유연, 더 복잡.

PAL: AND 프로그래밍 + OR 고정. PLA보다 저렴. 제약: AND 항 공유 불가, OR당 AND 수 고정. Registered PAL(PAL16R8)은 출력에 FF 포함 → 순차 논리 가능.

CPLD: 여러 PAL + 프로그래밍 가능 인터커넥트. Logic Block + Interconnect + IOB.

4. FPGA

1985년 Xilinx 최초 도입. 구성: CLB(LUT+FF) + Programmable Interconnect + IOB.

LUT — FPGA의 핵심

n-input LUT = 2^n SRAM entries + 2^n:1 MUX. 어떤 n-input 함수든 구현 가능.

```
  2-input LUT (AND 구현):
  SRAM[0]=0 (00→0)
  SRAM[1]=0 (01→0)  ──→┤4:1 MUX├──→ Out
  SRAM[2]=0 (10→0)     │        │
  SRAM[3]=1 (11→1)     └────────┘
                    A,B → select
  SRAM 내용 변경 → 어떤 2-input 함수든 가능 (OR: {0,1,1,1}, XOR:
{0,1,1,0})
```

Xilinx XC4000XL (클래식)

- CLB: 2개 4-input LUT + 1개 3-input LUT + 2개 DFF. 각 LUT는 RAM

으로도 사용 가능.
- Interconnect: Single/Double/Quad/Long Lines + PSM (6 pass transistors/point)
- IOB: 외부 핀 ↔ 내부 논리 인터페이스

Modern Xilinx 7 Series

- Slice: 4개 6-input LUT + 8개 FF
  - SLICEM: 조합/SRAM/shift register 가능
  - SLICEL: 조합만
- CLB: 2개 Slice (SLICEL+SLICEL 또는 SLICEL+SLICEM)
- DSP48 Slice: P = B×(A+D)+C 또는 P += B×(A+D) 전용 ALU
- Block RAM: 36Kb 단위. 1개 36Kb 또는 2개 18Kb. 인접 결합으로 64K×1 가능.
- Logic Cell: 전통적 4-input LUT+FF 1개. 6-input LUT 기준 약 1.6:1.

5. FPGA vs ASIC

항목	FPGA	ASIC
NRE	매우 낮음	매우 높음
Unit Cost	높음	낮음
Time to mkt	빠름	느림 (3~6개월)
설계 수정	재프로그래밍 가능	재제조 필요
성능/효율	ASIC 대비 낮음	최고
대량 생산	비쌈	저렴
소량 생산	저렴	비쌈
프로토타이핑	SW sim보다 훨씬 빠름	-

최근 트렌드: FPGA가 실제 제품에도 점점 사용 증가. 공정 미세화로 용량 증가, 설계 비용 절감, 구형 ASIC과 비교 시 에너지 효율 비슷. 범용 프로세서보다는 여전히 훨씬 좋은 성능.